
УЧЕБНИК · PYTHON · ВЕБ · ASYNC

«Скажи домашку»

Как устроен настоящий Python-проект: от webhook до деплоя



Учебник по проекту **alice-homework** · 14 глав · 15 разделов · код из репозитория без упрощений

Содержание

- 1 Домашка, которая не ждёт: почему быстрый ответ — целая архитектура
- 2 Дверь, в которую стучится Алиса: HTTP, JSON и webhook
- 3 Кухня со шкафчиками: зачем проект разбит на модули
- 4 Сейф с ключами: как хранить настройки и секреты
- 5 Рецепты без сети: чистая логика homework.py
- 6 Официант, который не ждёт: асинхронность и sgo.py
- 7 Холодильник на сервере: кеш в памяти cache.py
- 8 Курьер со свежими сэндвичами: фоновый воркер worker.py
- 9 Приёмная навыка: роутеры, фильтры и внедрение зависимостей
- 10 Дирижёр оркестра: настоящий веб-сервер skill.py
- 11 Страховочная сетка: как устроены 78 тестов
- 12 Городской адрес: как навык становится видимым для Алисы
- 13 Не финиш, а перекрёсток: куда можно вырасти
- 14 Глоссарий и шпаргалка
- ✦ Заключение

Как устроен настоящий Python-проект: от webhook до деплоя.

Этот учебник — подробный разбор реального (а не учебного) проекта: голосового навыка Яндекс Алисы «Скажи домашку», который читает домашнее задание из электронного дневника СГО. Мы пройдем по коду слоями: от HTTP-запроса Алисы до фоновой асинхронной воркера, и на каждом шаге разберём не только «что делают эти строки», но и «почему проект устроен именно так».

ДЛЯ КОГО. Вы уже знаете базовый синтаксис Python (переменные, функции, классы, списки, словари) и хотя бы запускали `python script.py`. Мы разберём то, чего обычно не хватает новичкам: веб, `async/await`, архитектуру модулей, тестирование, конфигурацию и деплой — всё на примерах одного проекта.

КАК ЧИТАТЬ. Каждая глава: сначала концепция простыми словами, потом — реальный код проекта по шагам, в конце — блок «Проверьте себя» с ответами. Код в учебнике — это код из репозитория, ничего не упрощено. Открывайте файлы проекта рядом и читайте учебник с кодом под рукой.

Глава 1. Домашка, которая не ждёт: почему быстрый ответ — целая архитектура

Вы уже написали не одну программу на Python: переменные, функции, списки — всё знакомо. А теперь представьте обратное: вместо скрипта, который вы сами запускаете в терминале, нужно построить сервис, который **живёт в интернете** и отвечает голосовому помощнику. Задача, казалось бы, та же — «прочитать домашку», а устройство совсем другое.

Эта глава — экскурсия по проекту. Мы разберём, что именно делает навык, из каких систем складывается один устный ответ и почему «просто ходить в дневник по каждому вопросу» не работает, как бы этого ни хотелось.

В ЭТОЙ ГЛАВЕ:

- что происходит между фразой «Алиса, что задали?» и голосовым ответом;
- почему платформа не даёт сервису больше 4,5 секунды;
- как из этого ограничения рождается архитектура: фоновый воркер + кеш;
- как запустить проект на своём компьютере и посмотреть на него вживую.

1.1. Один вечер из жизни навыка

Представьте: обычный вечер, завтра в школу, а домашнее задание записать никто не успел. Вы говорите: «Алиса, попроси навык Скажи домашку узнать, что задали». Через секунду слышите в ответ: «На завтра, в среду, два задания. Первое — алгебра: параграф 12, номера 5 и 6, с двумя вложениями. Второе — литература: прочитать главу 3. Удачи с уроками!»

Кажется, что ответ целиком родился в одну секунду. На самом деле за ним стоит цепочка из трёх программных систем:

1. **Яндекс.Диалоги** — платформа, на которой живёт навык. Она принимает голос, превращает его в текст (команду) и отправляет нашему сервису запрос по HTTP.

2. **Наш сервис** (`alice-homework`) — aiohttp-приложение на Python. Это «мозг» навыка: понимает команду, добывает данные, формирует текст ответа.
3. **СГО** (Система «Сетевой город», адрес `sgo.e-mordovia.ru`) — школьный электронный дневник. Из него навык берёт сами домашние задания.

Термин. *Скилл* (skills), или *навык* Яндекс Алисы, — это сторонний сервис, который подключается к платформе Яндекс.Диалоги. Платформа передаёт ему фразы пользователя, а он возвращает готовые ответы. Аналогия: Диалоги — телефонный оператор, навык — человек, который отвечает на звонки.

Простой, казалось бы, сценарий — «прочитать домашку» — таит главную сложность проекта. И сложность эта сидит не в школьном дневнике, а в **ограничениях платформы**:

- Алиса ждёт ответ от сервиса **не дольше 4,5 секунды**. Если сервис молчит дольше — навык считается сломавшимся, и пользователь слышит ошибку платформы.
- А каждый запрос в СГО (логин → чтение дневника → получение вложений) занимает **несколько секунд**. Логин, авторизация, несколько сетевых вызовов — всё это время не поместить в отведённые 4,5 секунды.
- Значит, запрашивать домашку «в момент вопроса» физически невозможно: сервис просто не успеет ответить вовремя.

ВОПРОСЫ ЛЮБОПЫТНЫХ.

— **Постойте, 4,5 секунды — это же прилично. Что там такого медленного?**

— Смотря для чего. Для человека 4,5 секунды — мгновение. Для программы — целая вечность. Но внутри этих секунд последовательно выполняются сетевые запросы к дневнику: авторизация, получение дневника, получение вложений. Каждый такой запрос — это поход в интернет и обратно, а сеть не любит, когда её торопят. Суммарно это легко превышает лимит.

— **То есть решить задачу «в лоб» — ходить в СГО на каждый вопрос — нельзя?**

— Нельзя, если мы хотим, чтобы навык всегда отвечал вовремя. Но выход есть, и он ровно в этой главе: доставать данные заранее.

Как же обойти ограничение? Решение — спрятать сетевую работу в **фоновый воркер**, который периодически ходит в СГО и складывает свежий результат в **кеш в памяти**. А вебхук (тот самый код, который обслуживает запросы Алисы) только читает кеш. Вопрос пользователя обслуживается за миллисекунды — данные уже готовы.

Это главный архитектурный урок проекта, к которому мы будем возвращаться в каждой главе: **разделение «медленной работы» и «быстрой выдачи» через общий кеш.**

1.2. Вся карта одной картинкой

Положите перед собой такую схему — это «карта сокровищ» всего учебника. Каждая следующая глава будет заполнять одну клетку этой карты.



Термин. Вебхук (webhook) — это URL, по которому одна система присылает другой события. В нашем случае Яндекс.Диалоги шлют POST-запрос с фразой пользователя на адрес `/alice`, а наш сервис отвечает готовым текстом. А подробнее о webhook — в главе 2.

Схема читается сверху вниз: пользователь говорит → платформа передаёт фразу → вебхук получает запрос → Dispatcher выбирает подходящий обработчик → тот читает кеш → текст ответа возвращается наверх. Слева внизу спрятана «кухня»: воркер в фоне наполняет кеш. Обратите внимание: путь обработки запроса и путь добычи данных не пересекаются. Запросы Алисы видят только быструю часть — кеш. Так и выполняется правило «отвечать за миллисекунды».

Важно. Кеш хранит **ровно один** результат (домашку на завтра) и время его загрузки. Это не «большая база данных», а маленькая коробочка, которую воркер перезаписывает каждый цикл. Отсюда — простота: нечего синхронизировать, одно место для чтения, одно место для записи.

1.3. Запускаем: теория встречается с практикой

Хватит абстракций — давайте увидим проект в деле. Выполните по шагам: клонируйте репозиторий, создайте окружение и установите зависимости.

BASH

```
git clone https://github.com/freeuser3/alice-homework.git
cd alice-homework

python -m venv .venv          # создать изолированное окружение
# Windows:
.venv\Scripts\activate
# Linux/macOS:
# source .venv/bin/activate

pip install -r requirements.txt
```

СОВЕТ. Виртуальное окружение (`venv`) — это отдельная папка с собственными версиями библиотек. Она не даёт проектам «съедать» друг друга: зависимости этого репозитория живут в `.venv` , а не в системном Python.

Файл `requirements.txt` закрепляет зависимости:

TEXT

```
aliceio==0.2.3
aiohttp>=3.9.0
netschoolapi-plus @ git+https://github.com/freeuser3/netschool-api-plus.git
pytest>=8.0.0
pytest-asyncio>=0.21.0
```

Обратите внимание на разницу между операторами `==` и `>=` :

- `aliceio==0.2.3` — **точная версия**. Библиотека для Яндекс.Диалогов молодая и часто меняет API; фиксируя версию, мы гарантируем, что проект работает с той версией, под которую написан.
- `aiohttp>=3.9.0` — **минимальная версия**: подойдёт любая новее 3.9.
- `netschoolapi-plus @ git+...` — библиотеку берём прямо **из GitHub-репозитория** (форка), потому что оригинальный пакет в PyPI не загружен или не подходит.

Теперь создайте конфиг и запустите:

BASH

```
cp config.example.json config.json # Windows: Copy-Item config.example.json config.json
# заполни config.json своими данными СГО
python -m alice_skill.skill
```

Запись `python -m alice_skill.skill` означает: запустить модуль `skill` внутри пакета `alice_skill` как программу (в нём есть блок `if __name__ == "__main__": main()` — мы разберём его в главе 10). Сервер слушает `host` и `port` из конфига (по умолчанию `127.0.0.1:8000`), путь вебхука — `/alice` .

ПОПРОБУЙТЕ САМИ [🔧]

1. После установки выполните `python -m pytest` — должны пройти все 78 тестов проекта.
Закрепите в памяти: тесты здесь — не формальность, а страховка, о них будет отдельная глава 11.
2. Откройте папку проекта и найдите файлы, которые соответствуют словам из схемы: `worker`, `cache`, `sgo`, `handlers`. Просто посмотрите, как они называются, — детали будут дальше.
3. Загляните в `config.example.json` и сверьте названия полей со схемой.

1.4. Итоги главы

- Навык — это сторонний сервис, подключённый к платформе Яндекс.Диалоги.
- Ответ пользователю складывается из трёх систем: платформа → наш сервис → СГО.
- Платформа ждёт ответ не дольше 4,5 секунды, а запрос в СГО занимает больше — поэтому «ходить в дневник по каждому вопросу» нельзя.
- Выход: фоновый воркер заранее кладёт результат в кеш, а обработчик запроса только читает кеш. Медленная работа и быстрая выдача разделены.
- `requirements.txt`: `==` фиксирует точную версию, `>=` — минимальную.
- Проект запускается командой `python -m alice_skill.skill`; webhook живёт по пути `/alice`.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Почему нельзя просто «ходить в СГО по каждому вопросу пользователя»?
2. Что означают `==` и `>=` в `requirements.txt`?
3. Какие три системы участвуют в ответе навыка?

Решения.

1. Запрос в СГО занимает несколько секунд, а платформа ждёт ответ не дольше 4,5 секунды. Успеть «в лоб» невозможно, поэтому данные добываются заранее.
2. `==` фиксирует точную версию библиотеки (под неё написан проект), `>=` допускает любую версию новее указанной.
3. Яндекс.Диалоги (платформа, принимает голос и шлёт запрос), наш сервис `alice-homework` (обрабатывает запрос), СГО (источник домашних заданий).

Глава 2. Дверь, в которую стучится Алиса: HTTP, JSON и webhook

В главе 1 вы узнали: Алиса присылает нашему сервису запрос и получает ответ. А что именно летит по проводам? Ответ почти разочаровывает: обычный текст по строгим правилам. Эти правила называются **HTTP** (HyperText Transfer Protocol) — и на них стоит весь веб: браузер → сервер, приложение → API, Алиса → навык.

Мы разберём именно наш случай — и поймём, почему вся эта схема сводится к одному простому механизму: **кто-то стучится в адрес, а мы отвечаем текстом**.

В ЭТОЙ ГЛАВЕ:

- из чего состоит HTTP-запрос и почему его уместно сравнить с письмом в конверте;
- чем **GET** отличается от **POST** и почему webhook — это всегда **POST** ;
- что именно присылает Алиса и на какое поле смотрят фильтры команд;
- по какому пути запрос доходит до нашего обработчика;
- как «постучать» в вебхук руками, не подключая Алису.

2.1. HTTP-запрос — это письмо в конверте

Когда Алисе говорят «что задали?», Диалоги упаковывают фразу в HTTP-запрос и отправляют нашему сервису. Представьте бумажное письмо: на конверте написан адрес и правила доставки, внутри — лист с самим содержанием. На самом низком уровне HTTP-запрос — просто текст из двух частей:

TEXT

```
POST /alice HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{"session": {...}, "request": {"command": "что задали", ...}}
```

Разберём «конверт» по строкам:

- **POST** — **метод (глагол)**: «прими эти данные и что-то с ними сделай». В отличие от **GET** — «дай мне данные». Глаголов немного: **GET**, **POST**, **PUT**, **DELETE** ... Для webhook почти всегда **POST**.
- **/alice** — **путь (адрес)**: в нашем приложении ровно один webhook-адрес.
- **HTTP/1.1** — **версия протокола**: формальность, но обязательная.
- **Host**, **Content-Type** — **заголовки** — метаданные о запросе (для кого, в каком формате тело).
- После пустой строки — **тело**: сам лист письма, JSON с данными о пользователе и его реплике.

Наш сервис обрабатывает запрос и отвечает — тоже текстом:

TEXT

```
HTTP/1.1 200 OK
Content-Type: application/json

{"response": {"text": "На завтра, в среду, два задания. ...", "end_session": false}, ...}
```

Код `200` означает «успех». Всё, что видит платформа, — этот JSON: текст ответа и пара служебных полей. Из него Алиса синтезирует голос.

ТЕРМИН. *Webhook* (от англ. *hook* — «крючок») — это адрес, на который внешняя система **САМА** шлёт HTTP-запросы, когда происходит событие. Нам не нужно «спрашивать» Алису и ходить к ней в гости — это она стучится в нашу дверь: пользователь что-то сказал → платформа позвала наш `/alice`.

ВОПРОСЫ ЛЮБОПЫТНЫХ.

— А в чём разница между `GET` и `POST`, если можно просто всегда слать запросы?

— По-хорошему каждая команда должна делать своё дело. `GET` просит сервер *показать* данные — у него нет побочных эффектов, его безопасно повторять сколько угодно. `POST` просит сервер *принять* данные и *сделать что-то* — обработать команду, изменить состояние. Диалоги присылают нам фразу пользователя, и это всегда `POST`: принять фразу и вернуть ответ.

— А если сервер ответит не `200`?

— Тогда навык считается отвечающим неправильно: пользователь услышит либо текст ошибки платформы, либо тишину. Поэтому в коде проекта такие сценарии перехватываются заранее (главы 9–10), а не отдаются на волю случая.

2.2. Два уровня одного сервера: `aiohhttp` и `aliceio`

В роли «сервера» выступает библиотека **`aiohhttp`** (ей соответствует `aiohhttp>=3.9.0` в requirements). Для работы с Яндекс.Диалогами поверх `aiohhttp` есть библиотека **`aliceio`** — она понимает формат запросов Алисы и умеет раздавать их по «роутерам» — обработчикам команд.

Два типа библиотек стоит различать с самого начала:

- **`aiohhttp`** — общий HTTP-фреймворк: принимает запросы, раздаёт их функциям-обработчикам, отдаёт ответы. Это «веб-сервер вообще».
- **`aliceio`** — специализированный слой для Яндекс.Диалогов: знает поля запроса Алисы (сессия, команда, тип запроса), умеет фильтровать команды и превращать ответы Python в нужный JSON. Это «веб-сервер именно для навыков».

АНАЛОГИЯ. `aiohhttp` — это универсальный почтальон: умеет приносить любые письма в любые дома. `aliceio` — курьер, который знает только один конверт (формат Алисы), зато знает его до мелочей: что написано на конверте, где указана команда и как должно выглядеть письмо в ответ.

Взгляните на начало `alice_skill/skill.py`:

```

from __future__ import annotations

import logging

import aiohttp.web as web
from aliceo import Dispatcher, Skill
from aliceo.webhook.aiohttp_server import (
    OneSkillAiohttpRequestHandler,
    setup_application,
)

from alice_skill.cache import HomeworkCache
from alice_skill.config import Config, load_config
from alice_skill.handlers.common import ERROR_TEXT, TIMEOUT_TEXT
from alice_skill.handlers.fallback import fallback_router
from alice_skill.handlers.homework import homework_router
from alice_skill.handlers.more import more_router
from alice_skill.handlers.start import start_router
from alice_skill.sgo import fetch_homework
from alice_skill.worker import PrefetchWorker

logger = logging.getLogger(__name__)

```

Список импортов читается как оглавление файла: сначала «инструменты» (aiohttp, aliceo), потом «свои» модули проекта. Уже по импортам видно, что соберёт глава 10: `Config`, `Dispatcher`, роутеры, `HomeworkCache`, `PrefetchWorker` и `fetch_homework` — все главные герои репозитория собраны в одной точке.

Две строки заслуживают отдельного слова:

- Строка `from __future__ import annotations` — маленькая деталь современного Python: откладывает «вычисление» аннотаций типов, позволяя писать аннотации вроде `Result | None` (см. главу 6). Пока достаточно знать, что она это делает.
- `logger = logging.getLogger(__name__)` — стандартный механизм логгирования. У каждого модуля свой логгер с именем `alice_skill.skill`, `alice_skill.sgo` и т.д. По логам сразу видно, из какого модуля пришло сообщение.

2.3. Что присылает Алиса: содержание письма

Теперь заглянем внутрь конверта. Настоящий запрос Алисы выглядит примерно так (упрощённо):

```
{
  "session": {
    "new": true,
    "session_id": "1a2b3c",
    "user_id": "user-123"
  },
  "version": "1.0",
  "request": {
    "type": "SimpleUtterance",
    "command": "что задали",
    "original_utterance": "Алиса, что задали"
  }
}
```

Это JSON — тоже текст по правилам, но уже для данных. Разберём поля, которые нам действительно важны:

- `session.new` — **новая ли сессия**: пользователь только что открыл навык — `true`, продолжает диалог — `false`. От этого зависит, поздороваться ли или отвечать по существу (глава 9).
- `session.session_id` — идентификатор диалога. Для простого навыка v1 мы его не храним: навык работает без состояния.
- `request.command` — нормализованная **команда**: текст, приведённый к нижнему регистру и без лишних слов. Именно на него смотрят фильтры (`F.command`) в главе 9.
- `request.type` — тип реплики: `SimpleUtterance` — обычная команда.

ВАЖНО. `command` — это не буквальная фраза, а очищенная версия: без регистра и «слов-паразитов». Поэтому навык устойчив к «Алиса, пожалуйста, скажи, что задали» — платформа сама отдаст короткое «что задали».

2.4. Как запрос доходит до нашего кода: путь по слоям

Полную цепочку удобно читать сверху вниз — как эстафету, в которой каждая станция делает одно маленькое дело:

1. `web.Application()` — aiohttp-приложение, «менеджер» всего веб-сервера.
2. `OneSkillAiohttpRequestHandler` — мостик между aiohttp и aliceio: кладёт запрос с пути `/alice` на обработку в Dispatcher.
3. `Dispatcher` — «распределитель» aliceio: разбирает JSON, создаёт объект `Message`, прогоняет его через роутеры-фильтры и вызывает нужный обработчик. Здесь же настроен `response_timeout=4.0` — тот самый лимит на обработку.
4. **Роутеры** (`start_router`, `homework_router`, `more_router`, `fallback_router`) — проверяют команду и решают, кто будет отвечать (глава 9).
5. **Обработчик** (например `handle_homework`) — формирует текст ответа через `answer_from_cache` (главы 7, 9).

Эстафета в коде выглядит так (функция целиком в `skill.py`, детально — глава 10):

```
def create_app(config: Config) -> web.Application:
    cache = HomeworkCache()
    worker = PrefetchWorker(
        fetch=lambda: fetch_homework(
            config.sgo.login, config.sgo.password, config.sgo.school,
        ),
        cache=cache,
        interval=config.prefetch_interval,
    )

    dp = Dispatcher(
        response_timeout=4.0,
        cache=cache,
        worker=worker,
    )

    # Router order matters: start → homework → more → fallback
    dp.include_router(start_router)
    dp.include_router(homework_router)
    dp.include_router(more_router)
    dp.include_router(fallback_router)
    ...
```

Три важных идеи, которые видно уже сейчас:

- **Роутеры подключаются в строгом порядке.** Сначала проверяется «новая сессия», затем — «команда про домашку», затем — «дальше», и только потом ловится всё остальное. Порядок имеет значение: отвечает **первый совпавший** фильтр.
- **cache и worker создаются заранее и «впрыскиваются» в Dispatcher.** Обработчики получают их через параметры — это внедрение зависимостей (глава 9).
- **Воркер привязан к данным из конфига** через `lambda` — функцию-заглушку, которая вызовет `fetch_homework(...)`. Подробно о `lambda` — в главе 6.

2.5. Стучимся в вебхук руками, не подключая Алису

Вебхук — это обычный HTTP-адрес. Значит, можно обойтись без самой Алисы и просто отправить запрос самостоятельно:

```
curl -X POST http://127.0.0.1:8000/alice \
  -H "Content-Type: application/json" \
  -d '{"session":{"new":true,"session_id":"1","user_id":"1"},"version":"1.0","request":
{"type":"SimpleUtterance","command":"что задали"}}'
```

- Запустите навык (`python -m alice_skill.skill`) до этой команды — сервер должен слушать порт.
- `curl` пошлёт POST на `http://127.0.0.1:8000/alice` с телом-JSON.
- Ответ придёт в том же JSON — тем самым, который Алиса превратит в голос.

СОВЕТ. В Windows команда `curl` в PowerShell может быть алиасом для `Invoke-WebRequest`. Если получите синтаксическую ошибку, вызывайте явно: `curl.exe -X POST ...`. А на маке/линуксе всё работает из коробки.

ПОПРОБУЙТЕ САМИ [🔗]

1. Постучите в вебхук командой из §2.5, но измените `command` на `"дальше"`, а потом на незнакомое слово (`"покажи рецепт борща"`). Сравните ответы.
2. Посмотрите логи сервера после каждого запроса: какие сообщения пишет логгер и видно ли имя модуля.
3. Намеренно сломайте JSON в `-d` (удалите одну скобку) и посмотрите, что ответит сервер и что окажется в логах.

2.6. Итоги главы

- HTTP-запрос — это текст из «конверта» (метод, путь, заголовки) и «листа» (тело).
- Webhook — адрес, на который внешняя система сама шлёт `POST`, когда случается событие. Алисе «звонить» не нужно — она стучится сама.
- `aihttp` — веб-сервер вообще, `aliceio` — веб-сервер именно для навыков Алисы.
- Алиса присылает JSON; команда для фильтров лежит в `request.command`, признак начала диалога — в `session.new`.
- Путь запроса: `aihttp` → `OneSkillAiohttpRequestHandler` → `Dispatcher` → роутеры → обработчик. Воркер в этот путь не входит — он наполняет кеш заранее.
- Вебхук проверяется руками через `curl`: `POST` с JSON на `/alice`.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Объясните своими словами, что такое webhook.
2. Чем `GET` отличается от `POST` и почему webhook использует `POST`?
3. В каком поле запроса лежит команда, на которую смотрят фильтры?
4. Что означают `session.new=true` и `session.new=false` для поведения навыка?

Решения.

1. Это адрес сервиса, на который внешняя система сама отправляет HTTP-запросы при наступлении события; нам не нужно опрашивать платформу.
2. `GET` просит показать данные (без побочных эффектов), `POST` просит принять данные и что-то сделать. Webhook передаёт фразу пользователя на обработку — это действие, поэтому `POST`.
3. В `request.command` — команда уже очищена платформой (регистр, лишние слова).
4. `true` — пользователь только что открыл навык (стоит поздороваться), `false` — продолжение диалога (отвечать по существу).

Глава 3. Кухня со шкафчиками: зачем проект разбит на модули

Представьте кухню, где тарелки, соль и сковородки лежат одной кучей посреди стола. Готовить можно — но только пока блюд мало. Теперь добавьте пару поваров, час пик и меню на шестьдесят персон. Куча на столе превращает кухню в хаос: никто не знает, где что лежит, и одно неосторожное движение сносит половину тарелок.

Код — та же кухня. До определённого размера «всё в одном файле» работает. Дальше начинается беда. Эта глава — о том, как проект раскладывает код по полкам и почему у каждой полки ровно одна работа.

В ЭТОЙ ГЛАВЕ:

- как называется болезнь «всё в одном файле» и в чём её потолок;
- из каких модулей состоит проект и за что отвечает каждый;
- почему здоровые зависимости не образуют «кругов»;
- чем «чистый» модуль отличается от «грязного»;
- как устроены пакеты и относительные импорты.

3.1. Болезнь «всё в одном файле»

Когда пишете первый проект, рука сама тянется сваливать всё в один `main.py`. У такого подхода есть потолок: через пару тысяч строк становится невозможно понять, где что лежит, что от чего зависит и что сломается при изменении.

Проект «Скажи домашку» — пример противоположной дисциплины: **один файл — одна ответственность**. Каждый файл отвечает на три вопроса: *что делает, как им пользоваться, от чего зависит*. Если на любой из них нельзя ответить одной фразой — файл делает слишком много.

ТЕРМИН. *Ответственность* (responsibility) модуля — это ровно одна роль, которую модуль играет в проекте. У `cache.py` одна работа — хранить результат. У `worker.py` одна работа — наполнять кеш по расписанию. Если у файла две работы, его стоит разделить.

3.2. Карта модулей: раскладываем по полкам

Вот как код раскладывается «по шкафчикам»:

```

alice-homework/
├─ alice_skill/
│   ├─ config.py          # загрузка настроек: файл + переменные окружения
│   ├─ homework.py       # чистая логика: дни, сбор домашки, голосовой текст
│   ├─ sgo.py            # клиент СГО: сеть, результаты, ошибки
│   ├─ cache.py          # кеш в памяти
│   ├─ worker.py         # фоновый воркер
│   ├─ skill.py          # сборка приложения, вебхук, точка входа
│   └─ handlers/        # роутеры и обработчики команд
│       ├─ common.py     # общий ответ через кеш + фразы
│       ├─ start.py      # новая сессия
│       ├─ homework.py   # «что задали» и подобное
│       ├─ more.py       # «дальше»
│       └─ fallback.py   # всё остальное
├─ tests/                # тесты (78 штук)
├─ config.example.json   # шаблон настроек (безопасный)
├─ requirements.txt
└─ README.md

```

Каждый модуль отвечает на три вопроса: **что делает, как им пользоваться, от чего зависит**. Проверим по каждому:

Модуль	Делает	Пользуются	Зависит от
<code>config.py</code>	Читает <code>config.json</code> + env, отдаёт объект <code>Config</code>	кто угодно	stdlib
<code>homework.py</code>	Чистая логика текста (без сети!)	<code>sgo.py</code>	stdlib + схемы netschoolapi
<code>sgo.py</code>	Ходит в СГО, отдаёт <code>HomeworkResult</code>	<code>worker.py</code>	<code>homework.py</code> , netschoolapi
<code>cache.py</code>	Хранит один результат	<code>worker</code> , <code>handlers</code>	<code>sgo.py</code> (тип)
<code>worker.py</code>	Фоновая загрузка по расписанию	<code>skill.py</code>	<code>cache</code> , <code>sgo</code>
<code>handlers/</code>	Реагируют на команды	<code>skill.py</code>	<code>cache</code> , <code>worker</code>
<code>skill.py</code>	Собирает всё вместе	<code>python -m alice_skill.skill</code>	все модули

Обратите внимание на **направление зависимостей** — это самое информативное место таблицы. Оно почти везде идёт «сверху вниз»: чистые и независимые модули (`config` , `homework`) лежат в основании, сетевой слой (`sgo`) выше, ещё выше — `cache` / `worker` , а собирает всё `skill.py` .

ВАЖНО. Если нарисовать зависимости стрелками, получится ациклическая цепочка без «кругов». Круг — это когда А зависит от В, а В — от А: менять такой код страшно, потому что любое изменение замыкается само на себя. В этом проекте стрелки всегда идут «вниз», поэтому каждую полку можно править отдельно. Отсутствие циклов — один из главных признаков здоровой архитектуры.

3.3. Чистая кухня и грязная кухня

Самое важное архитектурное правило проекта — разделение «**чистой**» логики и «**грязных**» эффектов:

- `homework.py` — «**чистый**» модуль. В нём **нет** сети, файлов, времени, случайности в неподконтрольном виде. Все входы — обычные аргументы: даты, список уроков, генератор случайных чисел. Такой модуль можно запустить и проверить за миллисекунды, не имея интернета.
- `sgo.py` — «**грязный**» модуль. Здесь сеть, логин, исключения, реальные часы. Прогнать его в тестах «по-настоящему» нельзя — не ходить же в чужой школьный сервер, — поэтому его тестируют через **заглушку-фейк** (глава 11).

Зачем такая строгость? Чем больше чистой логики, тем проще тестировать, рассуждать и находить баги. Сбой в сети — это не баг логики текста: они изолированы друг от друга.

АНАЛОГИЯ. «Чистая» логика — повар, который готовит по рецепту из заранее нарезанных продуктов. «Грязный» слой — закупщик, который тащит продукты с рынка. У закупщика сорвалась поставка — повар об этом узнает не мгновенно и работает с тем, что есть. В коде то же самое: сеть упала — `sgo` вернул «ошибку», а чистая логика спокойно продолжила обрабатывать данные, которые уже есть.

3.4. Пакеты и относительные импорты

`alice_skill` — это **пакет**: каталог с файлом `__init__.py`. Внутри — модули. Импорт вида `from alice_skill.sgo import fetch_homework` читается так: «из пакета `alice_skill`, модуля `sgo` возьми объект `fetch_homework`».

Внутри пакета удобно использовать **относительные импорты** (`.common` — «тот же пакет, модуль `common`»):

PYTHON

```
# alice_skill/handlers/start.py
from .common import answer_from_cache
```

Точка перед `common` обозначает «текущий пакет». Это важно: полное имя модуля — `alice_skill.handlers.common`, а не просто `common`. Так наш модуль не конфликтует с чужими библиотеками, у которых тоже может быть `common.py`.

СОВЕТ. Правило «один файл — одна ответственность» работает только вместе с осмысленными именами. Если имя модуля не укладывается в одно слово-ответ на вопрос «что он делает?» — скорее всего, ответственностей две.

ПОПРОБУЙТЕ САМИ [🔗]

1. Откройте `alice_skill/` и проговорите для каждого файла одну фразу «что он делает». Если фраза вышла длинной — подумайте, почему.
2. Возьмите таблицу зависимостей и нарисуйте стрелки на бумаге: найдите модуль, от которого зависит больше всего. Это «фундамент».
3. Откройте `skill.py` и сосчитайте, сколько импортов оттуда тянется в «свои» модули проекта. Почему именно он — «момент сборки»?

3.5. Итоги главы

- Один файл — одна ответственность; больше одной роли — повод разделить.
- Карта проекта: чистая логика, сеть, кеш, воркер, обработчики, сборка.
- Зависимости идут «сверху вниз» и не образуют кругов — это признак здоровья.
- «Чистые» модули (без сети, времени, файлов) легко тестируются; «грязные» покрываются заглушками.
- Пакет `alice_skill` даёт полные имена вроде `alice_skill.sgo`; относительные импорты (`.common`) работают только внутри пакета.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Перечислите ответственность каждого модуля одной фразой.
2. Почему `homework.py` называют «чистым», а `sgo.py` — «грязным»?
3. Что означает «направление зависимостей» и почему в этом проекте нет циклов?
4. Зачем каталогу нужен файл `__init__.py` (даже пустой)?

Решения.

1. `config` — настройки, `homework` — чистая логика текста, `sgo` — сетевой клиент СГО, `cache` — хранение одного результата, `worker` — фоновая загрузка, `handlers` — реакция на команды, `skill` — сборка приложения.
2. В `homework.py` нет сети, файлов и времени — все входы это аргументы; в `sgo.py` реальные сеть, логин, исключения и часы.
3. Модуль зависит только от «нижних» модулей; стрелки не образуют циклов — зависимости $A \rightarrow B$ и $B \rightarrow A$ нигде не встречаются, поэтому части правятся отдельно.
4. `__init__.py` превращает каталог в пакет Python — позволяет импортировать его модули по именам вроде `alice_skill.sgo` и даёт точку для общей инициализации.

Глава 4. Сейф с ключами: как хранить настройки и секреты

Каждый вечер навик открывает школьный дневник. Чтобы войти, ему нужны логин, пароль и название школы. Эти три значения — как ключи от квартиры: потеряете кольцо — злоумышленник войдёт в ваш дом. А ещё у навика есть «пропуск» — идентификатор `skill_id`, по которому Яндекс.Диалоги узнают именно наш навик.

Вопрос главы: где хранить ключи и настройки так, чтобы они не утекли в публичный репозиторий, но система при этом запускалась на любой машине?

В ЭТОЙ ГЛАВЕ:

- почему секреты нельзя класть в `git` и что делать вместо этого;
- как описать конфиг дата-классами и зачем `frozen=True`;
- что такое паттерн «env или файл» и как работает `or` в Python;
- почему функция конфигурации — мечта тестировщика.

4.1. Секреты: в `git` — только шаблон

У проекта три секрета: логин, пароль и школа ученика в СГО. Есть и настройка, которая не секрет, но тоже важна, — `skill_id` (идентификатор навыка в Яндекс.Диалогах).

ВАЖНО. Правило № 1: секреты не должны попадать в `git`-историю. Если пароль уехал в публичный репозиторий — он скомпрометирован навсегда. Даже если потом «почистить» историю, старые коммиты останутся в форках и клонах.

Поэтому в репозитории лежит только безопасный шаблон `config.example.json`:

JSON

```
{
  "sgo": {
    "login": "your_login",
    "password": "your_password",
    "school": "Your School Name"
  },
  "prefetch_interval": 1800,
  "host": "127.0.0.1",
  "port": 8000,
  "skill_id": "YOUR_SKILL_ID"
}
```

А настоящий `config.json` создаётся на машине разработчика и в `git` не попадает — он в `.gitignore`:

TEXT

```
config.json
__pycache__/
*.pyc
.venv/
*.egg-info/
.pytest_cache/
```

ВОПРОСЫ ЛЮБОПЫТНЫХ. Почему в `.gitignore` столько всего, а не только `config.json` ?

Посмотрите внимательно: там и `.venv` (окружение — сотни мегабайт, чужим не нужно), и `__pycache__` (сгенерированные `.pyc`-файлы), и `.pytest_cache` (служебные файлы тестов). В `git` попадает только то, что действительно нужно для сборки и чтения проекта: исходники, тесты, шаблоны конфигов.

4.2. Типобезопасная загрузка: дата-классы вместо словарей

Кто-то мог бы читать `config.json` как голый `dict` и писать `config["sgo"]["login"]`. Это работает, но таит ловушки: опечатка в строковом ключе не замечается до запуска. Проект идёт другим путём — объявляет **дата-классы**:

PYTHON

```
from __future__ import annotations

import json
import os
from dataclasses import dataclass
from pathlib import Path

@dataclass(frozen=True)
class SgoConfig:
    login: str
    password: str
    school: str

@dataclass(frozen=True)
class Config:
    sgo: SgoConfig
    skill_id: str
    prefetch_interval: int = 1800
    host: str = "127.0.0.1"
    port: int = 8000
```

Разбираем по кусочкам:

- `@dataclass` — декоратор из стандартного модуля `dataclasses`: он пишет за нас методы `__init__`, `__repr__`, `__eq__` для классов, которые **просто хранят данные**. `@dataclass(frozen=True)` дополнительно делает объект неизменяемым — после создания его поля нельзя перезаписать.

Зачем? Объект конфига курсирует по всему приложению; если никто не может его испортить, меньше шансов на баг.

- Поля с аннотациями: `login: str`, `prefetch_interval: int = 1800`. Второе поле — значение по умолчанию (30 минут, `127.0.0.1`, порт 8000).
- `SgoConfig` вложен в `Config` как поле `sgo` — композиция: «конфиг содержит конфиг СГО».

АНАЛОГИЯ. Словарь — это записная книжка, где всё ищется по словам. Дата-класс — это официальный бланк с напечатанными графами. На бланке сразу видно, какие графы есть; в записной книжке графу «логин» можно написать с любой опечаткой и никто не заметит, пока не придёт время её читать.

Почему классы лучше словаря? **Компилятор и IDE видят поля.** `config.sgo.login` проверяется статически и автодополняется; `config["sgo"]["login"]` — нет. А ещё с `frozen=True` опечатку вроде `config.sgo.loggin` нельзя ввести незаметно: попытка записи упадёт с ошибкой.

ТЕРМИН. *Композиция* — когда объект одного класса хранит внутри себя объект другого. Здесь `Config` «состоит из» `SgoConfig`. Композиция проще наследования, когда связь «часть — целое», а не «частный случай».

4.3. Приоритет настроек: файл или переменные окружения?

Теперь сама загрузка. Функция `load_config` умеет брать значения из двух источников и решает, какой важнее:

```
def load_config(path: str | Path = "config.json") -> Config:
    with open(path, encoding="utf-8") as f:
        raw: dict = json.load(f)

    sgo_raw: dict = raw.get("sgo", {})
    sgo = SgoConfig(
        login=os.environ.get("SGO_LOGIN") or sgo_raw.get("login", ""),
        password=os.environ.get("SGO_PASSWORD") or sgo_raw.get("password", ""),
        school=os.environ.get("SGO_SCHOOL") or sgo_raw.get("school", ""),
    )
    if not (sgo.login and sgo.password and sgo.school):
        raise ValueError(
            "sgo.login, sgo.password, sgo.school must be set in config.json "
            "or via SGO_LOGIN / SGO_PASSWORD / SGO_SCHOOL env vars"
        )

    skill_id = os.environ.get("SKILL_ID") or raw.get("skill_id", "")
    if not skill_id:
        raise ValueError("skill_id must be set in config.json or SKILL_ID env var")

    prefetch = int(os.environ.get("PREFETCH_INTERVAL") or raw.get("prefetch_interval", 1800))
    host = raw.get("host", "127.0.0.1")
    port = int(raw.get("port", 8000))

    return Config(sgo=sgo, skill_id=skill_id, prefetch_interval=prefetch, host=host, port=port)
```

Идеи здесь важные и общие:

1. **Паттерн «env или файл»:** `os.environ.get("SGO_LOGIN") or sgo_raw.get("login", "")`. В Python `or` возвращает **первое «правдивое» значение**. Если переменная окружения задана и непуста — берём её. Иначе — значение из файла. Так на локальной машине можно держать пароль в `config.json`, а на сервере — в системных переменных окружения, не дублируя файлы.
2. **Ранняя проверка с понятной ошибкой.** Если логин/пароль/школа пустые или нет `skill_id` — поднимаем `ValueError` **сразу при старте**, а не в середине работы. Принцип fail-fast: «упади громко и рано, а не тихо и поздно».
3. **Дефолт через `.get()`:** `raw.get("prefetch_interval", 1800)` — значение по умолчанию, если ключа нет в JSON.
4. `int(...)` приводит строку/число к `int` — в JSON интервал может быть числом, а переменная окружения всегда строка, поэтому `int()` необходим.

ВАЖНО. Обратите внимание на тонкость: `host` и `port` **НЕ** имеют env-переменных. Это сознательная дисциплина, а не забывчивость: если настраивать переменными всё подряд, настройки «разъезжаются». Переменные окружения получают только те значения, которые реально меняются между окружениями: секреты и интервал предзагрузки.

4.4. Как `load_config` тестируется

Чистая функция без сети — мечта тестировщика. Смотрите тесты (`tests/test_config.py`):

```
def test_load_config_valid(tmp_path):
    path = _write_config(tmp_path, {
        "sgo": {"login": "u", "password": "p", "school": "s"},
        "skill_id": "sid",
        "prefetch_interval": 600,
        "host": "0.0.0.0",
        "port": 9000,
    })
    cfg = load_config(path)
    assert isinstance(cfg, Config)
    assert cfg.sgo.login == "u"
    assert cfg.skill_id == "sid"
    assert cfg.prefetch_interval == 600
    assert cfg.host == "0.0.0.0"
    assert cfg.port == 9000
```

Как это работает: `tmp_path` — **фикстура pytest**: временный каталог, который pytest создаёт для каждого теста и сам удаляет. В него тест кладёт свой `config.json`, чтобы не трогать настоящий.

Проверку приоритета env-переменных делает фикстура `monkeypatch`:

```
def test_load_config_env_overrides(tmp_path, monkeypatch):
    path = _write_config(tmp_path, {
        "sgo": {"login": "u", "password": "p", "school": "s"},
        "skill_id": "sid",
    })
    monkeypatch.setenv("SGO_LOGIN", "env_user")
    monkeypatch.setenv("SGO_PASSWORD", "env_pass")
    monkeypatch.setenv("SGO_SCHOOL", "env_school")
    monkeypatch.setenv("SKILL_ID", "env_sid")
    cfg = load_config(path)
    assert cfg.sgo.login == "env_user"
    assert cfg.sgo.password == "env_pass"
    assert cfg.sgo.school == "env_school"
    assert cfg.skill_id == "env_sid"
```

`monkeypatch.setenv` временно ставит переменные окружения и гарантированно откатывает их после теста. Красота в том, что мы проверяем **реальный** приоритет «env выше файла», а не придуманный.

ПОПРОБУЙТЕ САМИ [🔗]

1. Запустите `python -m pytest tests/test_config.py -q` — все тесты зелёные: этому модулю не нужны ни интернет, ни сторонние библиотеки, ни конфиг.
2. Откройте `config.example.json` и `config.py` рядом: найдите в коде все места, где упоминается `prefetch_interval`, и проверьте, что дефолт одинаковый.
3. Добавьте в `.gitignore` строку `*.local.json` и положите файл `config.local.json`: убедитесь, что `git status` его больше не показывает.

4.5. Итоги главы

- Секреты — в git нельзя; в репозитории живёт только шаблон `config.example.json`.
- `.gitignore` прячет не только секреты, но и мусор сборки (`.venv`, `__pycache__`, кеши).
- Дата-классы с `frozen=True` дают типобезопасный, неизменяемый конфиг.
- Паттерн «env или файл» через `or` позволяет одной функцией обслужить и локаль, и сервер.
- Fail-fast: ошибки конфигурации вылетают при старте, а не в середине работы.
- `tmp_path` и `monkeypatch` превращают загрузку конфига в полностью тестируемую функцию.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Что означает `@dataclass(frozen=True)` и зачем это для конфига?
2. Объясните, как работает `os.environ.get("X") or raw.get("x", "")`.
3. Почему секреты лежат в `.gitignore`, а шаблон — в репозитории?
4. Что делает `int(...)` для значения из переменной окружения?

Решения.

1. Класс становится неизменяемым: поля нельзя перезаписать после создания. Это защищает объект конфига, который ходит по всему приложению.
2. `or` возвращает первое «правдивое» значение: если переменная окружения задана и непуста — берётся она; иначе — значение из файла (или пустая строка).
3. Шаблон нужен, чтобы любой мог понять структуру настроек; настоящие секреты создаются на машине разработчика и не должны попасть в историю.
4. Приводит значение к целому числу: env-переменная — это всегда строка, а интервал должен стать `int`.

Глава 5. Рецепты без сети: чистая логика `homework.py`

Теперь самое вкусное место учебника: здесь «сырые» данные дневника СГО превращаются в **человеческий голосовой текст** — фразу вроде «На завтра, в среду, два задания». И, как на кухне у хорошего повара из главы 3, здесь царит чистота: ни сети, ни файлов — только функции и данные.

Почему это лучший модуль для учёбы? Потому что в нём нет никакой «магии среды»: вы можете открыть его и прочитать как книгу рецептов. Давайте научимся читать такой код — он легче, чем кажется.

В ЭТОЙ ГЛАВЕ:

- зачем выносить «магические строки» в именованные константы;
- как таблицы слов в списках превращают число в пропись;
- почему «11 заданий» — ловушка, о которую спотыкаются все;
- как регулярные выражения вычищают HTML в две строки;
- почему «правильные» функции сами управляют временем и случайностью.

5.1. Константы и словари

Начало модуля:

PYTHON

```
import datetime
import random
import re
from typing import Optional

from netschoolapi_plus.schemas import Diary

HOMEWORK_TYPE = "Домашнее задание"
EMPTY_TEXT = "На завтра ничего не задали. Можно отдохнуть!"
CLOSING = "Удачи с уроками"
```

Разбираем:

- `HOMEWORK_TYPE` — в СГО тип задания — это строка `"Домашнее задание"`. Мы сравниваем с ней тип каждого задания. Вынос значения в именованную константу (а не «магическую строку» в коде) — привычка, которая спасает от опечаток: одна буква «а» в двух местах — и сравнение тихо сломается.
- `EMPTY_TEXT`, `CLOSING` — фразы, из которых собирается ответ.

ТЕРМИН. *Магическая строка* — литерал, который повторяется в коде без имени. Строка `"Домашнее задание"` в десяти местах рано или поздно в одном из них опечатается. Константа `HOMEWORK_TYPE` сводит все места к одному.

Дальше — большие списки русских слов. Это «таблица», зашитая в код:

```

_UNITS = [
    None, "одно", "два", "три", "четыре", "пять",
    "шесть", "семь", "восемь", "девять",
]
_TEENS = [
    "десять", "одиннадцать", "двенадцать", "тринадцать", "четырнадцать",
    "пятнадцать", "шестнадцать", "семнадцать", "восемнадцать", "девятнадцать",
]
_TENS = [
    None, None, "двадцать", "тридцать", "сорок", "пятьдесят",
    "шестьдесят", "семьдесят", "восемьдесят", "девяносто",
]

```

Обратите внимание на первый элемент `None` в `_UNITS` и `_TENS`. Индексация списка начинается с нуля, поэтому на позиции «два» стоит индекс 2: `_UNITS[2] == "два"`, а `_UNITS[0]` просто нет — там `None`. Зачем такой сдвиг? Чтобы индекс «1» (цифра «один») совпадал с номером элемента: вместо `_UNITS[n-1]` писать `_UNITS[n]`. Просто, но удобно. Та же «сдвинутая» структура повторяется для творительного падежа и для дней недели.

ВОПРОСЫ ЛЮБОПЫТНЫХ. А если число нулевое, не вылетит ли `_UNITS[0]` ?

Нет: до обращения дело не дойдёт. Взгляните на `number_to_words` в следующем разделе — для `n == 0` там отдельная ветка, возвращающая `"ноль"`. Сдвиг списка — это *способ* адресации, а не защита от нуля.

5.2. Функции превращения числа в слово

```

def number_to_words(n: int) -> str:
    if n == 0:
        return "ноль"
    if 1 <= n <= 9:
        return _UNITS[n]
    if 10 <= n <= 19:
        return _TEENS[n - 10]
    if 20 <= n <= 99:
        tens, units = divmod(n, 10)
        if units == 0:
            return _TENS[tens]
        unit_word = "одно" if units == 1 else _UNITS[units]
        return f"{_TENS[tens]} {unit_word}"
    return str(n)

```

- `divmod(n, 10)` возвращает пару (частное, остаток): `divmod(23, 10) == (2, 3)`. Так мы получаем десятки и единицы.
- Особый случай: `21 → "двадцать одно"` (а не «двадцать один») — потому что слово «задание» среднего рода. Сравните: 21 — «двадцать одно», 22 — «двадцать два». Поэтому для единиц, равных 1, подставляем «одно».

- `return str(n)` — fallback: числа от 100 вряд ли встретятся (столько заданий за день не бывает), но функция должна вести себя разумно — лучше число цифрами, чем ошибка.

По той же схеме `number_to_words_instrumental` строит творительный падеж: «с тремя вложениями», «с двадцатью пятью вложениями».

СОВЕТ. Такие «переводчики» числа в слова — отличная разминка для программиста: в них есть границы диапазонов (1-9 , 10-19 , 20-99), особые случаи и fallback. Программа со сложной логикой, но без сети — лучший полигон для тестов.

5.3. Склонение существительного: `plural_count`

Русский язык склоняет существительные после числительных по хитрым правилам: 1 задание , 2 задания , 5 заданий , 11 заданий , 21 задание . Функция:

PYTHON

```
def plural_count(n: int) -> str:
    n100 = n % 100
    n10 = n % 10
    if 11 <= n100 <= 19:
        return "заданий"
    if n10 == 1:
        return "задание"
    if 2 <= n10 <= 4:
        return "задания"
    return "заданий"
```

- `n % 100` — последние две цифры, `n % 10` — последняя цифра. Это деление по модулю: остаток от деления.
- Классическая ловушка русского языка — **11–14**: «11 заданий», хотя на вид «один» + «задание». Правило «2–4 → задания» перестаёт работать, поэтому сначала проверяем десятки 11–19 и возвращаем «заданий».

БЕЗ ПАНИКИ. Правило 11–19 — известная засада не только в коде, но и в устной речи: иностранец скажет «двенадцать задание», и все поймут, но звучит странно. Порядок проверок в функции — не случайность: сначала «широкое» исключение (11–19), потом узкие правила. Нарушите порядок — и 12 даст неверный «два + задания».

5.4. Вложения: `attachments_phrase`

PYTHON

```
def attachments_phrase(n: int) -> str:
    if n == 0:
        return ""
    if n == 1:
        return ", с вложением"
    if n == 2:
        return ", с двумя вложениями"
    return f", с {number_to_words_instrumental(n)} вложениями"
```

Полное покрытие трёх случаев: 0 — ничего не добавляем, 1 — «с вложением», 2 — «с двумя», 3+ — «с тремя/четырьмя/...» в творительном падеже. Обратите внимание: запятая включена в строку с начала (", с вложением"), чтобы просто конкатенировать с результатом.

АНАЛОГИЯ. Функция как приставка к слову: вернула пусто — слово не изменилось, вернула «с вложением» — приставили хвостик. Такой «хвостик», уже содержащий запятую, позволяет собирать фразу без логики «нужна ли запятая здесь?».

5.5. Чистка контента: регулярные выражения

Задание в СГО может содержать HTML-теги. Пользователю они не нужны — нужен чистый текст:

PYTHON

```
def _clean_content(content: str) -> str:
    text = re.sub(r"<[^>]+>", " ", content)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

- `re.sub(pattern, замен, текст)` — замена всех вхождений по регулярному выражению.
- `r"<[^>]+>"` — «угловая скобка, затем один или более любых символов, кроме `>`, затем `>`». То есть весь тег заменяется пробелом.
- `r"\s+"` — «один и более пробельных символов» → один пробел.
- `.strip()` — обрезает пробелы по краям.

Регулярные выражения — мощный, но опасный инструмент; в таких маленьких дозах («выкинуть теги», «сжать пробелы») они идеальны.

ТЕРМИН. *Regex*, или регулярное выражение, — это «шаблон строки», в котором символы и спецобозначения описывают множество совпадений: `<[^>]+>` описывает любой тег, `\s+` — любую цепочку пробелов. Python вызывает это через модуль `re`.

5.6. «На завтра или на среду»: `_opener`

PYTHON

```
def _opener(target: datetime.date, today: datetime.date) -> str:
    if target == today + datetime.timedelta(days=1):
        return "На завтра, " + _WEEKDAY_PREP[target.weekday()]
    return "На " + _WEEKDAY_ACC[target.weekday()]
```

- `datetime.timedelta(days=1)` — «промежуток в один день»; `today + timedelta(1)` — завтрашняя дата.
- `target.weekday()` — номер дня недели (понедельник = 0, воскресенье = 6). Он служит индексом в списках `_WEEKDAY_ACC` (винительный падеж: «на понедельник») и `_WEEKDAY_PREP` (предложный: «в понедельник», но «во вторник»).

5.7. Сбор домашки: `next_school_day` и `collect_homework`

Дневник СГО — это объект `Diary` со списком «дней» (`schedule`), у каждого дня — список уроков, у урока — список заданий:

PYTHON

```
def next_school_day(diary: Diary, start: datetime.date) -> Optional[datetime.date]:
    for day in sorted(diary.schedule, key=lambda d: d.day):
        if day.day >= start and day.lessons:
            return day.day
    return None
```

- `sorted(..., key=lambda d: d.day)` — сортируем дни по дате. `key=` — функция, по значению которой сравниваем: для каждого дня берём его `day` (дату).
- Условие: день не раньше `start` и в нём есть уроки (`day.lessons` не пуст — так отсекаются выходные и каникулы с пустыми днями).
- `Optional[datetime.date]` — аннотация «дата или `None`». `Optional[X] = X | None`. Возврат `None` означает «учебного дня в обозримом будущем нет».

```
def collect_homework(diary: Diary, target: datetime.date) -> list[dict]:
    result: list[dict] = []
    for day in diary.schedule:
        if day.day != target:
            continue
        for lesson in sorted(day.lessons, key=lambda l: l.number):
            for assignment in lesson.assignments:
                if assignment.type == HOMEWORK_TYPE:
                    result.append({
                        "day": day.day,
                        "number": lesson.number,
                        "subject": lesson.subject,
                        "content": assignment.content,
                        "assignment_id": assignment.id,
                        "attachments": [],
                    })
    return result
```

- Три вложенных цикла: дни → уроки → задания. Если день не целевой — `continue` : переходим к следующему дню (экономия и читаемость).
- Уроки сортируются по номеру (`lesson.number`) — порядок уроков по расписанию.
- В результат попадают только задания с типом «Домашнее задание».
- Результат — список **словарей**. Каждый элемент — «плоский» контейнер с ключами: день, номер, предмет, текст, id (для вложений), пустой список вложений (заполнит сетевой слой).

ВОПРОСЫ ЛЮБОПЫТНЫХ. Зачем возвращать `list[dict]` , а не дата-класс, как в главе 4?

Это осознанный компромисс: `collect_homework` — «сборочный цех», который отдаёт данные дальше по конвейеру, и в `format_for_voice` словари читаются по ключам. Строгая типизация здесь была бы наряднее, но словари — ровно то, что удобно передавать «пачкой» между модулями. Правило «один файл — одна ответственность» важнее «один тип — на каждую фигуру данных».

5.8. Сборка голосового текста: `format_for_voice`

Финальная, самая показательная функция:

```

def format_for_voice(
    entries: list[dict],
    target: datetime.date,
    today: datetime.date | None = None,
    rng: random.Random | None = None,
) -> str:
    if not entries:
        return EMPTY_TEXT
    today = today or datetime.date.today()
    rng = rng or random.Random()
    count = len(entries)
    count_phrase = f"{number_to_words(count)} {plural_count(count)}"
    opener = f"_{opener(target, today)}, {count_phrase}."
    parts: list[str] = [opener]
    for i, entry in enumerate(entries):
        subject = entry["subject"].lower()
        body = _clean_content(entry["content"])
        att_count = len(entry.get("attachments", []))
        body += attachments_phrase(att_count)
        if count == 1:
            prefix = "Только"
        elif i == 0:
            prefix = "Первое —"
        elif i == count - 1 and count >= 3:
            prefix = rng.choice(LAST_CONNECTORS)
        elif i == 1 and count == 2:
            prefix = "Второе —"
        else:
            prefix = rng.choice(MIDDLE_CONNECTORS)
        parts.append(f"{prefix} {subject}: {body}.")
    parts.append(f"{CLOSING}!")
    return " ".join(parts)

```

Что стоит разобрать:

- **Инъекция времени и случайности** (`today=...`, `rng=...`): функция по умолчанию берёт «сегодня» и случайный генератор сама, но может принять их извне. Это делает её детерминированной **в тестах**: передаём фиксированную дату и `random.Random(42)` — и результат всегда одинаковый. Дизайнерский паттерн dependency injection в миниатюре (подробнее — глава 9).
- Каскад `if/elif` — правила выбора связок. Одно задание — «Только». Два — «Первое», «Второе». Три+ — «Первое», далее случайная средняя связка, последний пункт — из `LAST_CONNECTORS`. Заметьте порядок условий: случай «последний пункт» проверяется до случая «второй из двух».
- `enumerate(entries)` — пара (`номер`, `элемент`): число и сам элемент в цикле.
- `.lower()` у предмета — приводим «Алгебра» к «алгебре» для естественного звучания фразы.
- Итог — список строк `parts`, соединённых через `" ".join(parts)`. Обработка текста «накапливанием кусочков» — типичный паттерн сборки.

Пример вывода, который мы уже видели в главе 1:

На завтра, в среду, два задания. Первое — алгебра: параграф 12, номера 5 и 6, с двумя вложениями. Второе — литература: прочитать главу 3. Удачи с уроками!

5.9. Как это тестируется

Тесты лежат в `tests/test_homework.py`. Идея — вызывать `format_for_voice` с «игрушечным» списком записей и проверять, что текст ровно такой, как ожидали:

PYTHON

```
def test_format_for_voice_two_items():
    target = datetime.date(2026, 9, 21) # понедельник
    today = datetime.date(2026, 9, 20)  # воскресенье
    entries = [
        {"subject": "Алгебра", "content": "Упр. 5", "attachments": []},
        {"subject": "Русский язык", "content": "Упр. 10", "attachments": []},
    ]
    text = format_for_voice(entries, target, today=today)
    assert "два задания" in text
    assert "Первое — алгебра: Упр. 5." in text
    assert "Второе — русский язык: Упр. 10." in text
```

Так как логика чистая и детерминированная (мы передаём `today` — что считать «сегодня» — а для трёх и более пунктов ещё и `rng` с фиксированным seed, как `random.Random(42)`), тесты стабильны: они не зависят от времени суток или случая.

ПОПРОБУЙТЕ САМИ [🔗]

1. Запустите `python -m pytest tests/test_homework.py -q` — этому модулю не нужны ни сеть, ни `aliceio`, поэтому он зелёный на любой машине.
2. Откройте репозиторий и найдите `_WEEKDAY_ACC` и `_WEEKDAY_PREP`. Убедитесь, что там есть и «во вторник», и «на вторник».
3. В консоли выполните `divmod(100, 10)` и `divmod(101, 10)` — что изменится?

5.10. Итоги главы

- Чистая логика без сети и файлов — читается и тестируется за мгновения.
- Константы вместо магических строк защищают от опечаток.
- Таблицы-списки со «сдвинутым» индексом превращают числа в слова.
- В `plural_count` порядок проверок решает: 11–19 проверяется раньше правил 1 и 2–4.
- Regexp-чистка (`<[>]+>`, `\s+`) убирает HTML из контента в две строки.
- Параметры `today` / `rng` делают функцию детерминированной — тесты стабильны.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Почему `_UNITS[0]` — это `None`? Что изменится, если убрать «сдвиг»?
2. Объясните, что делает `divmod(23, 10)` и зачем здесь этот вызов.
3. В чём ловушка чисел 11–14 в `plural_count`?
4. Что делает `re.sub(r"<[^>]+>", " ", content)`?
5. Зачем `format_for_voice` принимает `today` и `rng` параметрами, а не всегда берёт «сегодня» и случайность сам?

Решения.

1. Индексация списков начинается с нуля, а нам нужно, чтобы номер «1» (один) совпадал с индексом 1. Уберите сдвиг — не будет `_UNITS[1] == "один"` и индекс придётся везде смещать на единицу.
2. Возвращает кортеж `(2, 3)` — десятки и единицы числа. Дальше по ним берутся слова из `_TENS` и `_UNITS`.
3. У 11–14 «хвост» выглядит как единица/две-четыре, но склонение у них как у «заданий». Без явной проверки 11–19 правило «2–4 → задания» дало бы ошибку.
4. Заменяет каждый HTML-тег одним пробелом; вместе с `\s+` это даёт чистый текст.
5. Чтобы функция была детерминированной: тесты передают фиксированные «сегодня» и генератор и получают предсказуемый результат в любой день суток.

Глава 6. Официант, который не ждёт: асинхронность и

sgo.py

Представьте ресторан с одним официантом. Гость заказал стейк. Официант смотрит, как стейк жарится двадцать минут, — и только потом идёт принимать заказ у соседнего стола. Знакомо? Это **синхронный** путь: пока ждём — стоим.

А теперь другой официант: принял заказ, отнёс его на кухню и тут же пошёл к следующему столу. Через двадцать минут забрал готовый стейк и отнёс. За то же время он обслужил десяток гостей. Это **асинхронный** путь: ждать — не значит стоять.

До этой главы мы не трогали сеть. Сеть — это как кухня, где блюдо готовят сотни миллисекунд, а то и секунды. Если сервер навыка станет «стоять у кухни», пока СГО жарит ответ, он не сможет обслужить других пользователей. Вся сетевая часть проекта — `sgo.py`, `worker.py`, `skill.py` — построена на том самом «втором официанте».

В ЭТОЙ ГЛАВЕ:

- что такое корутины, `await` и цикл событий;
- какой контракт обещает сетевой слой: `HomeworkResult` ;
- как `try/except/finally` превращает любую сетевую ошибку в вежливый ответ;
- почему сбой вложений не должен ронять весь ответ;
- зачем `fetch_homework` передают воркеру через `lambda` .

6.1. Почему сети мало одного потока

Классический Python-код выполняется **синхронно**: строка за строкой, и пока компьютер ждёт сетевого ответа, программа стоит. Сеть — это медленно: ответ от сервера может идти сотни миллисекунд. Если сервер монополюно ждёт ответа от СГО, он не может ответить другим пользователям — все «зависают» в очереди к одному официанту.

Асинхронный код решает это иначе: когда нужно подождать сеть, он говорит «буду готов дальше, а вы пока займитесь другими делами». Одно ядро процессора обслуживает много «одновременных» операций, кооперативно переключаясь между ними.

Пара ключевых понятий:

- **Корутина** — функция, объявленная через `async def` . Она не выполняется сразу при вызове, а возвращает объект-корутину. Запускается она только в цикле событий (event loop).
- `await` — точка, где корутина может «уснуть», отдавая управление циклу. Пока она спит, цикл выполняет другие корутины.
- `asyncio` — стандартная библиотека. Функциями `asyncio.sleep` , `asyncio.create_task` и другими мы управляем этим переключением.

ТЕРМИН. Цикл событий (event loop) — «диспетчер» асинхронной программы. Он хранит список готовых корутин и решает, кого будить следующим. Пока одна корутина спит на `await` , диспетчер передаёт управление другой.

В нашем проекте всё, что касается сети, асинхронное: `netschoolapi-plus` — асинхронный клиент, `aiohttp` — асинхронный сервер, `worker` — асинхронная задача.

6.2. Контракт: `HomeworkResult`

Прежде чем писать сетевой код, проект определяет **формат ответа**, который функция `fetch_homework` вернёт всем заинтересованным (воркеру, кешу, обработчикам):

PYTHON

```
from typing import Literal

@dataclass
class HomeworkResult:
    status: Literal["ok", "empty", "error"]
    target_date: datetime.date | None
    text: str
    error: str | None = None
```

- `Literal["ok", "empty", "error"]` — тип «только эти три строки». `Literal` — заготовленная аннотация из `typing`. Так IDE и инструменты (муру) предупреждают, если где-то появится `status = "success"` — такой строки нет в «словаре».
- Значение по умолчанию — только у `error` (`None`). `target_date` и `text` задаются явно в каждом `HomeworkResult(...)`: так не получится случайно «забыть» заполнить текст ответа.

АНАЛОГИЯ. `HomeworkResult` — это бланк ответа, который заполняет курьер и отдаёт заказчиком. Курьер не знает, как заказчик покажет текст пользователю, а заказчик не знает, как курьер добывал текст. Все договорились только о бланке.

Это классический подход «одна функция знает результат, остальные — формат». Потребители не знают, как результат добывается, а производитель не знает, как его покажут пользователю.

6.3. Главная функция: `fetch_homework`

PYTHON

```
async def fetch_homework(
    login: str,
    password: str,
    school: str,
    *,
    url: str = SGO_URL,
    now: datetime.date | None = None,
) -> HomeworkResult:
    ns = NetSchoolAPI(url, default_requests_timeout=None)
    try:
        await ns.login(login, password, school)
        today = now or datetime.date.today()
        target = today + datetime.timedelta(days=1)
        diary = await ns.diary(start=target, end=target + datetime.timedelta(days=7))
        day = next_school_day(diary, target)
        if day is None:
            return HomeworkResult(status="empty", target_date=None, text=EMPTY_TEXT)
        entries = collect_homework(diary, day)
        for entry in entries:
            entry["attachments"] = await _assignment_attachment_names(
                ns, entry["assignment_id"]
            )
        text = format_for_voice(entries, day, today=today)
        status = "ok" if entries else "empty"
        return HomeworkResult(status=status, target_date=day, text=text)
    except Exception as exc:
        logger.exception("fetch_homework failed")
        return HomeworkResult(
            status="error", target_date=None, text=ERROR_TEXT, error=str(exc),
        )
    finally:
        try:
            await ns.logout()
        except Exception:
            pass
```

Разбор сюжета:

1. Создаём клиента `NetSchoolAPI`. Параметр `default_requests_timeout=None` снимает внутренний таймаут клиента: ограничение времени в этой цепочке задаём мы (в воркере и таймере обработки), а не библиотека.
2. `try / except / finally` — три блока с разными ролями:
 - `try` — основная работа (сеть, дневник, текст);
 - `except Exception as exc` — **перехватываем всё**: любая ошибка сети, логина или неверного пароля превращается в `HomeworkResult(status="error")` с текстом `ERROR_TEXT` (вежливая фраза «Не получилось заглянуть в дневник...», объявлена в этом же модуле). Никакое исключение не

должно просочиться наружу и уронить воркер. Стек ошибки пишется в лог (`logger.exception`), а внутренний текст (`str(exc)`) сохраняется в поле `error` — для диагностики;

- `finally` — выполняется **в любом случае**: и при успехе, и при ошибке. Здесь — выход (`await ns.logout()`), но разлогин обернут в собственный `try/except: pass` : при оборванной сети он сам может упасть, и это не должно перебить основной результат.
- 3. `await ns.login(...)` , `await ns.diary(...)` — асинхронные вызовы. Пока они выполняются, сервер обслуживает других клиентов.
- 4. `today/now` : как и в `homework.py` , сделана **инъекция времени** (`now=None` → берём `date.today()`). В тестах код работает с фиксированной «сейчас».
- 5. `target = today + 1 день` — «завтра». Окно чтения — от `target` до `target + 7 дней` , чтобы `next_school_day` нашёл ближайший учебный день, даже если завтра воскресенье или каникулы.
- 6. Статус результата вычисляется по факту: `"ok"` , если записи есть, `"empty"` , если их нет. А если `next_school_day` вернул `None` (учебного дня в окне нет) — сразу возвращается `"empty"` с `EMPTY_TEXT` .
- 7. Поток данных нагляден: клиент → дневник → учебный день → список записей → голосовой текст.

БЕЗ ПАНИКИ. `except Exception` ловит всё — даже ошибку, которую вы не предвидели. Это почти всегда разумно на границе системы (сеть — граница), но внутри приложения такой «широкий» `except` — антипаттерн: он прячет баги. Здесь он на своём месте: сетевой слой обещает никогда не падать, что бы ни случилось.

6.4. Вложения: пусть будет пусто, когда не вышло

PYTHON

```
async def _assignment_attachment_names(ns: NetSchoolAPI, assignment_id: int) -> list[str]:
    try:
        attachments = await ns.attachments(assignment_id)
    except Exception:
        return []
    return [a.name for a in attachments]
```

Маленькая, но важная деталь: **вложения не критичны**. Если вложений получить не удалось — мы не обрушим весь ответ, а вернём пустой список (слово о вложениях в текст просто не добавится). Особого логирования здесь нет — второстепенная неудача тихо проглатывается.

ТЕРМИН. Такая стратегия называется *graceful degradation* — «деградация без отказа»: система продолжает работать, даже когда второстепенная деталь вышла из строя. Это не лень, а осознанный выбор: лучше ответить пользователю без списка вложений, чем не ответить вовсе.

6.5. Гарантированный выход из системы

Разлогин аккуратно «защищён» отдельным `try/except` внутри `finally` :

PYTHON

```
finally:
    try:
        await ns.logout()
    except Exception:
        pass
```

Разлогин может упасть (сеть уже оборвалась) — это не должно мешать основному потоку вернуть результат. Поэтому сбой при выходе просто игнорируется. Мелочь, но именно такие «спасательные круги» отличают боевой код от учебного.

ВОПРОСЫ ЛЮБОПЫТНЫХ. А не утечка ли сессии, если `logout()` так легко бросать?

Нет: если сеть уже оборвалась, сессия умрёт сама. А если сеть жива — `logout()` сработает и закроет сессию штатно. Игнорирование сбоя здесь — «последняя страховка», а не пренебрежение.

6.6. Зачем `lambda` в фабрике воркера

Вспомним строчку из главы 2:

PYTHON

```
worker = PrefetchWorker(
    fetch=lambda: fetch_homework(
        config.sgo.login, config.sgo.password, config.sgo.school,
    ),
    cache=cache,
    interval=config.prefetch_interval,
)
```

`lambda` — безымянная функция, которая **вызывает** `fetch_homework` с уже «прописанными» аргументами из конфига. Зачем это нужно?

- Воркеру безразлично, откуда берётся результат, — ему нужна просто «асинхронная функция, которая вернёт `HomeworkResult`».
- Конфиг существует только в `skill.py`; воркер живёт своей жизнью. Именно `lambda` связывает их, не втаскивая конфиг внутрь воркера.

Это ещё один аккуратный приём разделения ответственности. (В более сложных проектах вместо лямбды использовали бы класс-провайдер, но для нашей задачи лямбда — самый простой и честный вариант.)

ПОПРОБУЙТЕ САМИ [🔪]

1. Откройте `alice_skill/sgo.py` и найдите константу `SGO_URL`. Что будет, если передать `fetch_homework` несуществующий `url`? Подсказка: посмотрите, куда уходит `url` и что вернётся.
2. Проговорите вслух («на пальцах») судьбу вложения: сеть отвалилась прямо на третьем файле. Что вернёт `_assignment_attachment_names` и что попадёт в текст?
3. Найдите все возвраты `HomeworkResult` в `fetch_homework` и раскройте их: где будет `status="ok"`, где `"empty"`, а где `"error"`.

6.7. Итоги главы

- Асинхронность — «ждать, не стоя»: `await` отдаёт управление циклу событий.
- `HomeworkResult` — контракт: `status`, `target_date`, `text`, `error`.
- `try/except/finally` превращает любую сетевую поломку в вежливый `"error"`.
- Некритичные вложения работают по принципу graceful degradation.
- Разлогин в `finally` защищён собственным `try/except`, чтобы не перебить ответ.
- `lambda` связывает воркер с конфигом без внедрения конфига в воркер.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Чем `async def`-функция отличается от обычной? Что вернёт `fetch_homework(...)` в первый момент до `await`?
2. Какие роли у блоков `try`, `except`, `finally` и что было бы, если бы `finally` не было?
3. Почему `_assignment_attachment_names` при ошибке возвращает `[]`, а не поднимает исключение?
4. Что такое `Literal["ok", "empty", "error"]` и чем его можно заменить?

Решения.

1. `async def` создаёт корутину: при вызове она не выполняется, а возвращает объект-корутину, который запускается только в цикле событий.
2. `try` — основная работа, `except` — превращение любой ошибки в `"error"`-ответ, `finally` — гарантированный разлогин. Без `finally` разлогин при исключении был бы пропущен, и сессия могла «повиснуть».
3. Вложения не критичны: лучше ответить без списка файлов, чем потерять весь ответ. Это graceful degradation.
4. Это тип «только одна из трёх строк». Заменить можно на `str` — тогда ошибки в статусе перестанут замечаться статическим анализом.

Глава 7. Холодильник на сервере: кеш в памяти `cache.py`

Представьте, что вы заказали в кафе сэндвич, а повару нужно сначала слетать за хлебом в булочную. Пока он ездит, вы ждёте. Хороший повар привозит хлеб заранее, с утра, и держит готовые сэндвичи в холодильнике: гость пришёл — сэндвич достали и подали за секунду.

Наш навык — то же кафе. Платформа Яндекса даёт на ответ **4,5 секунды**, а добыча домашки из СГО — это многие секунды сетевых `await` ов. Если отвечать «свежим запросом в СГО», повар будет ездить в булочную на каждом ответе — и опоздает.

Решение напрашивается само собой: **делайте работу заранее, а при вопросе просто выдавайте готовый результат**. Для этого и нужен кеш — наш «холодильник».

В ЭТОЙ ГЛАВЕ:

- из чего состоит кеш на один результат: всего 26 строк;
- почему временные метки в кеше живут в UTC;
- чем `is_stale` похожа на запасной пожарный выход;
- как тесты «отматывают время», не трогая часы;
- почему `webhook` ни разу не создаёт сетевого запроса.

7.1. Холодильник на один сэндвич: код

PYTHON

```
from __future__ import annotations

import datetime
from dataclasses import dataclass

from alice_skill.sgo import HomeworkResult

@dataclass
class HomeworkCache:
    _result: HomeworkResult | None = None
    _fetched_at: datetime.datetime | None = None

    def get(self) -> HomeworkResult | None:
        return self._result

    def set(self, result: HomeworkResult) -> None:
        self._result = result
        self._fetched_at = datetime.datetime.now(tz=datetime.timezone.utc)

    def is_stale(self, ttl: datetime.timedelta) -> bool:
        if self._result is None or self._fetched_at is None:
            return True
        return (
            datetime.datetime.now(tz=datetime.timezone.utc) - self._fetched_at > ttl
        )
```

Всего 26 строк — а это полноценный, легко проверяемый компонент. Разберём.

7.2. Атомарность кеша

- Кеш хранит **один** `HomeworkResult` — только «домашку на завтра». Несколько пользователей читают один и тот же кеш: СГО шлёт данные школы, а не отдельно «для Пети» и «для Маши». Это осознанный выбор — кеш один на приложение.
- Поле `_fetched_at` — момент загрузки в UTC. Почему именно UTC? Пишем временные метки в «мировом» времени, а не в локальном часовом поясе сервера: иначе при переносе сервера в другой пояс кеш стал бы «свежим» или «протухшим» по-другому, а логика перестала бы быть предсказуемой.
- Приватные поля с подчёркиванием (`_result`, `_fetched_at`) — договорённость: «это внутренности, наружу — только через методы».

ТЕРМИН. *UTC* — «всемирное координированное время», ось отсчёта для всех часовых поясов мира. Логика, которая сравнивает «сейчас» и момент загрузки, должна использовать одну ось; если каждый часовой пояс меряет по-своему, «свежесть» станет относительной.

7.3. `is_stale` : запасной выход

По замыслу воркер обновляет кеш каждые `prefetch_interval` секунд (по умолчанию 1800 — 30 минут). Но если СГО лежит, обновление проваливается, и кеш мог бы остаться «вечным». Метод `is_stale(ttl)` позволяет проверять свежесть.

По сути, это заготовка: сегодня воркер хранит результат бессрочно (что подтверждают и тесты), но механизм контроля свежести — на месте, и его легко включить. Хороший признак кода: **интерфейс опережает текущие потребности**.

СОВЕТ. «Запасной выход» — это метод, которого сейчас никто не использует, но он сделан правильно и протестирован. Когда через полгода понадобится TTL — достаточно будет подключить его в одном месте. Такой код живёт дольше, чем его первое применение.

7.4. Тестирование холодильника

`tests/test_cache.py` проверяет кеш кучей маленьких тестов:

PYTHON

```
def test_set_and_get():
    cache = HomeworkCache()
    result = _ok_result("На завтра одно задание.")
    cache.set(result)
    assert cache.get() is result
    assert cache.get().text == "На завтра одно задание."
```

и про свежесть:

PYTHON

```
def test_stale_when_empty():
    cache = HomeworkCache()
    assert cache.is_stale(timedelta(minutes=30)) is True

def test_not_stale_immediately_after_set():
    cache = HomeworkCache()
    cache.set(_ok_result())
    assert cache.is_stale(timedelta(hours=1)) is False

def test_stale_after_ttl():
    cache = HomeworkCache()
    cache.set(_ok_result())
    # Rewind the timestamp to simulate an old fetch
    cache._fetched_at = datetime.datetime.now(
        tz=datetime.timezone.utc,
    ) - timedelta(hours=2)
    assert cache.is_stale(timedelta(hours=1)) is True
```

Смотрите, как время проверяется без подмены часов: тест просто **вручную откатывает** `_fetched_at` на два часа назад (поле «приватно» только по договорённости), и `is_stale` честно сравнивает. Никаких «перемоток» времени — только данные.

ВОПРОСЫ ЛЮБОПЫТНЫХ. Разве тест не «лезет в чужое поле» — ведь `_fetched_at` приватное?

Приватность в Python — договорённость, а не запрет (вспомните про подчёркивания из прошлого раздела). В тестах мы сознательно нарушаем её, чтобы проверить поведение без ожидания реальных двух часов. Это стандартный приём: тест владеет внутренностями точно так же, как владеет фикстурами.

7.5. Связка: воркер → кеш → webhook

Теперь вся картинка по частям складывается:

ТЕКСТ

```
Воркер: every 30 мин -> fetch_homework() -> HomeworkResult -> cache.set(result)
Webhook: при вопросе -> cache.get() -> если есть ok/empty -> отдаём текст
```

Дешёвое и критичное звено — именно webhook. Он **ни разу** не создаёт сетевого запроса. Поэтому отвечает почти мгновенно и укладывается в 4,5 секунды.

ПОПРОБУЙТЕ САМИ [🔗]

1. Запустите `python -m pytest tests/test_cache.py -q`. Посмотрите, сколько крошечных тестов описывают поведение кеша в разных состояниях.
2. В `tests/test_cache.py` найдите хелпер `_ok_result`. Что он возвращает и как сокращает каждый тест?
3. Подумайте: зачем `set` запоминает `_fetched_at`, если `get` его не использует нигде? Найдите, кто вызывает `is_stale` (например, `grep -r is_stale`).

7.6. Итоги главы

- Кеш на один результат — 26 строк, от которых зависит скорость всех ответов.
- Один `HomeworkResult` на приложение: данные школы у всех пользователей общие.
- Метки времени — только UTC: «свежесть» не зависит от часового пояса сервера.
- `is_stale(ttl)` — готовый и протестированный «запасной выход» на будущее.
- Тесты отматывают `_fetched_at` вручную — без подмены системных часов.
- Webhook отвечает без сети; сетевую работу делает только воркер заранее.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Почему время кеша хранится в UTC, а не в локальном часовом поясе?
2. Что означает `HomeworkResult | None` в аннотации поля `_result`?
3. Зачем метод `set` запоминает `_fetched_at`?

Решения.

1. UTC — единая ось отсчёта: сравнение «сейчас» и момента загрузки даёт один и тот же результат в любом часовом поясе сервера.
2. Это аннотация «или — или»: поле хранит либо `HomeworkResult`, либо `None` (когда кеш ещё пуст). Читается как «результат или ничего».
3. Чтобы `is_stale` могла посчитать возраст данных: без метки невозможно отличить «свежий результат» от «результата недельной давности».

Глава 8. Курьер со свежими сэндвичами: фоновый воркер

`worker.py`

В прошлой главе у нас появился холодильник — кеш. Но холодильник не наполняется сам. Нужен курьер, который заранее привозит свежие сэндвичи: пришёл утром, положил запас, оставил визитку «позвоните, если понадобится срочно» — и с каждым получасом проверяет, не пора ли доложить.

Этот курьер — **фоновый воркер**: постоянно живущая асинхронная задача, которая раз в `prefetch_interval` секунд ходит в СГО и обновляет результат. Посмотрим на его класс.

В ЭТОЙ ГЛАВЕ:

- как воркер получает «голую» функцию загрузки и живёт в неведении о СГО;
- зачем `start` сначала наполняет кеш, а только потом запускает цикл;
- как флаг `_running` не даёт двум обновлениям налезть друг на друга;
- что происходит, если сеть упала, — а что было бы без перехвата;
- чем `refresh_now` отличается от `start` и почему он не `async` ;
- как аккуратно `stop` гасит фоновый цикл.

8.1. Курьер: код

PYTHON

```
from __future__ import annotations

import asyncio
import contextlib
import logging

from alice_skill.cache import HomeworkCache
from alice_skill.sgo import ERROR_TEXT, HomeworkResult

logger = logging.getLogger(__name__)

class PrefetchWorker:
    def __init__(
        self,
        fetch,
        cache: HomeworkCache,
        interval: float,
    ) -> None:
        self._fetch = fetch
        self._cache = cache
        self._interval = interval
        self._task: asyncio.Task | None = None
        self._running: bool = False

    async def start(self) -> None:
        await self._refresh_once()
        self._task = asyncio.create_task(self._loop())

    async def _loop(self) -> None:
        while True:
            await asyncio.sleep(self._interval)
            await self._refresh_once()

    def refresh_now(self) -> None:
        """Fire-and-forget refresh; safe to call from handlers."""
        asyncio.create_task(self._refresh_once())

    async def _refresh_once(self) -> None:
        if self._running:
            return
        self._running = True
        try:
            result = await self._fetch()
        except Exception as exc:
            logger.exception("prefetch failed unexpectedly")
            result = HomeworkResult(
                status="error", target_date=None, text=ERROR_TEXT, error=str(exc),
```

```

    )
    finally:
        self._running = False
        self._cache.set(result)

    async def stop(self) -> None:
        if self._task is not None:
            self._task.cancel()
            with contextlib.suppress(asyncio.CancelledError):
                await self._task

```

8.2. Разбор цеха доставки

- **Конструктор хранит зависимости** (`fetch` , `cache` , `interval`). Воркер не имеет понятия о конфигурации и СГО — он получает «голую» функцию `fetch` , которая `async def` , возвращающая `HomeworkResult` (или падающая). Так класс легко тестировать подделкой функции.
- **start** — первичная загрузка **исходного** значения (чтобы кеш не был пуст и первый же вопрос не провалился), затем постоянный цикл. `asyncio.create_task` — запускает корутину «параллельно» без ожидания её окончания; задача живёт, пока приложение живо.
- **Защита от пересечения обновлений:** флаг `_running` . Пока одна операция обновления выполняется, вторая не лезет, даже если кто-то вызвал `refresh_now()` во время сетевого ожидания. В асинхронном коде нет гонок потоков, но гонки между корутинами возможны, и `if self._running:` `return` справляется с ними.
- **Перехват ошибок сети:** `self._fetch()` может падать (СГО недоступен), но из `_refresh_once` исключение не «вылетит наружу»: на его место подставляется `HomeworkResult(status="error")` с фразой `ERROR_TEXT` (берётся из `sgo.py`). Воркер живёт, следующий цикл попробует снова — кеш просто получит «ошибку».
- **refresh_now** — «fire-and-forget»: немедленное обновление без ожидания. Обратите внимание: это **обычный** метод, не `async` , — он ничего не ждёт, а лишь создаёт фоновую задачу (`asyncio.create_task`). Вызывается из `webhook` , когда ответ рановат («результата ещё нет»): приложение отвечает пользователю фразой «преждевременно», а воркер в фоне обновляет кеш — следующий запрос («дальше») получит данные.
- **stop** — корректная остановка: отменяет периодическую задачу и **дожидается её завершения** (`await self._task`), проглатывая `CancelledError` через `contextlib.suppress` . Вызывается при завершении приложения (в `shutdown` -хуке, глава 10), чтобы не «молотить» цикл бесконечно в процессе выключения.

ТЕРМИН. *Fire-and-forget* — «запустил и забыл»: мы не ждём результата операции. Создали задачу — и цикл событий сам решит, когда её выполнить. Так устроен `refresh_now` : он не блокирует вызов, а лишь ставит задачу «в очередь».

8.3. Как тестировать курьера без СГО

Красота этого подхода — тесты можно писать без настоящей сети: воркеру неважно, кто работает «под» `fetch` . В `tests/test_worker.py` сетевой вызов заменяют `AsyncMock` -ом:

```

from unittest.mock import AsyncMock

@pytest.mark.asyncio
async def test_start_runs_fetch_once_and_sets_cache():
    cache = HomeworkCache()
    fetch = AsyncMock(return_value=_ok_result())
    worker = PrefetchWorker(fetch=fetch, cache=cache, interval=86400)

    await worker.start()
    await asyncio.sleep(0)
    assert cache.get() is not None
    assert cache.get().text == "homework"
    fetch.assert_awaited_once()
    await worker.stop()

```

- `AsyncMock(return_value=...)` — готовая асинхронная «подделка» из `unittest.mock`: при `await fetch()` мгновенно возвращает сфабрикованный `HomeworkResult`. Сеть ни разу не касается теста.
- `await worker.start()` запускает воркер по-настоящему; `await asyncio.sleep(0)` просит цикл событий сделать «шаг», чтобы фоновая задача успела отработать.
- `fetch.assert_awaited_once()` — проверка: функция вызывается ровно один раз.
- `await worker.stop()` в конце аккуратно останавливает периодический цикл.

ВОПРОСЫ ЛЮБОПЫТНЫХ. Как проверить, что воркер действительно «спит между заходами», не дожидаясь тридцати минут в реальном времени?

В тестах берут маленький интервал (`interval=0.01`), делают `await asyncio.sleep(...)` побольше интервала и считают вызовы `fetch`. Событийный цикл честно показывает число обновлений за время ожидания — сеть имитируется, а календарь не нужен.

ПОПРОБУЙТЕ САМИ [🔗]

1. Откройте `tests/test_worker.py`: какие сценарии покрыты? Проговорите каждый тест одним предложением.
2. Что случится, если заменить `if self._running: return` на `while self._running`? Почему возврат, а не ожидание — правильный выбор для fire-and-forget?
3. Найдите вызов `worker.stop()` в `skill.py` и проследите путь от клавиатуры до отмены задачи.

8.4. Итоги главы

- Воркер наполняет кеш по расписанию, а не по запросу — отсюда скорость ответов.
- Зависимости (`fetch`, `cache`, `interval`) подаются извне — воркер «не знает» о СГО.
- `start` сначала наполняет кеш «здесь и сейчас», потом заводит цикл.
- `_running` защищает от наложения двух обновлений.
- Любая ошибка `fetch` становится `HomeworkResult(status="error")` — воркер живёт.
- `refresh_now` — fire-and-forget: обычный метод, ставящий задачу в очередь.
- `stop` отменяет задачу и дожидается её, проглатывая `CancelledError`.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. В чём разница между `start()` и `refresh_now()`, если оба вызывают `_refresh_once`?
2. Зачем нужен флаг `_running`?
3. Что произойдёт с воркером, если `fetch` упадёт с исключением? А если бы исключение не перехватывалось?
4. Почему воркер получает `fetch` извне, а не знает про СГО?

Решения.

1. `start` — полный запуск: первичное наполнение кеша плюс бесконечный периодический цикл. `refresh_now` — одноразовое обновление «в фоне», без ожидания результата и без создания цикла.
2. Чтобы две копии `_refresh_once` не выполнились одновременно: пока одна ждёт сеть, вторая не должна лезть в то же обновление.
3. Исключение превращается в `HomeworkResult(status="error")`, кеш получает его, воркер продолжает работать. Без перехвата задача `_loop` умерла бы вместе с исключением, и обновления прекратились.
4. Разделение ответственности: воркер умеет «обновлять кеш», а не «ходить в СГО». Источник данных подставляется снаружи — так же легко подставить тестовую заглушку.

Глава 9. Приёмная навыка: роутеры, фильтры и внедрение зависимостей

Из главы 2 мы знаем: запрос Алисы приходит к нам как пакет JSON. Но как пакет превращается в конкретное действие — поздороваться, рассказать домашку или подсказать? Это работает как приёмная в большой школе: есть секретари, у каждого — список вопросов, за которые он отвечает, и чёткая очерёдность. Спросили про домашку — секретарь «домашка» отвечает; спросили непонятно что — в дело идёт секретарь «всё остальное».

В этой главе мы соберём «лицевую» часть навыка — слои-роутеры из `aliceio` и фильтры, а заодно разберёмся, откуда обработчики берут кеш и воркер.

В ЭТОЙ ГЛАВЕ:

- что такое роутер и как декоратор регистрирует обработчик;
- как фильтры `F.command.contains(...)` распознают «что задали»;
- зачем в цепочке последний «всепрощающий» обработчик-fallback;
- что делает общий хелпер `answer_from_cache`;
- как внедрение зависимостей кормит обработчиков кешем и воркером;
- что означает `response_timeout=4.0`.

9.1. Секретари и регистрационные карточки: обработчики

Каждый «ответственный за команду» модуль в `handlers/` объявляет свой роутер:

PYTHON

```
# handlers/homework.py
from aliceio import F, Router
from aliceio.types import Message, Response

from .common import answer_from_cache

homework_router = Router(name="homework")

HOMEWORK_FILTER = (
    F.command.contains("домашк")
    | F.command.contains("дз")
    | F.command.contains("что задали")
    | F.command.contains("задани")
)

@homework_router.message(HOMEWORK_FILTER)
async def handle_homework(message: Message, cache, worker) -> Response:
    return answer_from_cache(cache, worker)
```

Разбор:

- `Router(name="homework")` — роутер — это «коробка» обработчиков. Имя помогает в логах и отладке.

- `@homework_router.message(...)` — декоратор, регистрирующий обработчик. Аргумент внутри — фильтр.
- `F.command` — обращение к полю `command` у сообщения. `F.command.contains(...)` — фильтр «команда содержит эту подстроку». `|` между условиями — логическое ИЛИ. Скобки важны: приоритет операторов. Фильтр срабатывает, если команда содержит хотя бы одно из: «домашк», «дз», «что задали», «задани».
- `HOMEWORK_FILTER` вынесен в константу: его удобно документировать и, при желании, переиспользовать в тестах.

Обработчик — `async def` с параметрами `message` (объект из `aliceio`), а также `cache` и `worker`, — и возвращает `Response`. Ключевой момент: `cache` и `worker` приходят **сами** (см. §9.3), мы ничего не создаём. Ответ уже готовит общий хелпер `answer_from_cache(cache, worker)` — обработчик лишь передаёт ему зависимости.

ТЕРМИН. *Роутер* (router) — контейнер обработчиков с фильтрами. Запрос «стучится» в роутеры по очереди: каждый проверяет свои фильтры, и первый, чей фильтр сработал, обрабатывает запрос.

9.2. Остальные секретари на месте

`handlers/start.py` — новая сессия:

PYTHON

```
@start_router.message(F.session.new)
async def handle_start(message, cache, worker) -> Response:
    return answer_from_cache(cache, worker)
```

`F.session.new` — совпадение по полю `session.new == True`: пользователь только что открыл навык. Он **не выигрывает** конкурс у остальных: если это новая сессия и пользователь сразу спросил «что задали», обработчик `homework` перехватит команду первым (порядок включения роутеров: `start` → `homework` → `more` → `fallback`). Задача `handle_start` — поздороваться с готовым ответом после «запусти навык».

`handlers/more.py` — «дальше»:

PYTHON

```
@more_router.message(F.command == "дальше")
async def handle_more(message, cache) -> Response:
    result = cache.get()
    if result is not None:
        return Response(text=result.text)
    return Response(text=HINT_TEXT)
```

Кнопка «дальше»/«ещё» просто повторяет то, что сейчас лежит в кеше — **любой статус**, даже `error`: пользователю важнее получить хоть какой-то ответ, чем молчание. Кеша нет вообще — подсказка.

`handlers/fallback.py` — всё остальное:

PYTHON

```
@fallback_router.message()
async def handle_fallback(message) -> Response:
    return Response(text=HINT_TEXT)
```

Фильтр пустой — обработчик ловит **любой** запрос, который не попал выше (последний в цепочке). Это «окно», гарантирующее: каждый запрос получит ответ. Отсутствие такого «fallback» в навыке давало бы платформе «не знаю, что ответить» — а это плохо для навыка.

СОВЕТ. Fallback — это ваша страховка от тишины. Лучше честно сказать «Я умею рассказывать домашку», чем промолчать. Пользователь хотя бы поймёт, что навык жив.

`handlers/common.py` — общий мост:

PYTHON

```
from aliceio.types import Response

from alice_skill.cache import HomeworkCache
from alice_skill.worker import PrefetchWorker

PREMATURE_TEXT = "Секунду, заглядываю в дневник. Скажи «дальше»."
HINT_TEXT = "Я умею рассказывать домашку. Скажи «что задали»."
TIMEOUT_TEXT = "Не успела посмотреть в дневник. Скажи «что задали» ещё раз."
ERROR_TEXT = "Что-то пошло не так. Попробуй, пожалуйста, ещё раз."

def answer_from_cache(cache: HomeworkCache, worker: PrefetchWorker) -> Response:
    result = cache.get()
    if result is not None and result.status in ("ok", "empty"):
        return Response(text=result.text)
    worker.refresh_now()
    return Response(text=PREMATURE_TEXT)
```

Это сердце скорости навыка:

- Кеш полон и статус хороший → мгновенный ответ текстом из кеша. Никакой сети.
- Иначе → «рановато»: вызывается `worker.refresh_now()` (фоновая подзагрузка) и пользователю возвращается «Секунду, заглядываю в дневник. Скажи „дальше“.».
- `result.status in ("ok", "empty")` — проверка набора: `"error"` из кеша не годится для ответа про домашку — пробуем заново.

АНАЛОГИЯ. `answer_from_cache` — как холодильник в кафе из главы 7: гость пришёл — сэндвич в холодильнике, подали мгновенно. Сэндвича нет — повару дали знать, и гостю сказали «минуту, доложим». Кнопка «дальше» — это «вернуться за сэндвичем через полминуты».

Все служебные фразы (`PREMATURE_TEXT` , `HINT_TEXT` , `TIMEOUT_TEXT` , `ERROR_TEXT`) определяются один раз в `common.py` и импортируются оттуда, где нужны, — единый источник правды.

9.3. Внедрение зависимостей (DI)

Взгляните ещё раз на обработчик:

```
PYTHON
```

```
async def handle_homework(message: Message, cache, worker):
```

Откуда приходят `cache` и `worker`? В `skill.py` мы строим Dispatcher и передаём именно эти объекты:

```
PYTHON
```

```
dp = Dispatcher(response_timeout=4.0, cache=cache, worker=worker)
```

`Dispatcher` — «контейнер зависимостей»: сохраняет эти объекты и при вызове обработчика ищет среди параметров обработчика те, что знает, и подставляет их. `message` приходит из конвейера Алисы, `cache` и `worker` — из контейнера. Это и есть паттерн **внедрение зависимостей** (Dependency Injection, DI):

- Обработчики **не создают** зависимостей сами (нет `HomeworkCache()` внутри каждого).
- Обработчики только **декларируют**, что им нужно.
- Один экземпляр кеша и воркера живёт на всё приложение и передаётся всем обработчикам — консистентность.

ТЕРМИН. Внедрение зависимостей — приём, при котором объект «просит» свои зависимости извне, а не создаёт их сам. Школьная аналогия: учитель не носит с собой классный журнал — ему его приносят; а если журнал один на школу, никто не потеряет записи.

Выгоды: тестируемость (передаёшь в тесте фейковый кеш), отдельность (`handler` не знает, как строить воркер), центральная точка создания (легко поменять воркер на другой — изменится только `skill.py`).

9.4. Зачем `response_timeout`

`response_timeout=4.0` — это не время ответа сети, а **лимит на выполнение обработчика** (в секундах). Обработчики отвечают из кеша — миллисекунды. Если времени не хватило, срабатывает зарегистрированный `@dp.timeout()`-обработчик (глава 10), и платформа получит фразу `TIMEOUT_TEXT` — навык не «зависнет» молча.

ВАЖНО. `response_timeout` — это «страховой трос»: в норме он никогда не срабатывает, но если случится что-то неожиданное — пользователь получит ответ, а не тишину. На платформе Алисы тишина хуже любого текста.

ПОПРОБУЙТЕ САМИ [🔗]

1. Посмотрите `handlers/` в репозитории: у какого роутера пустой фильтр? Почему он должен стоять последним?
2. Добавьте в `HOMEWORK_FILTER` ещё одно условие — например, `F.command.contains("уроки")`. Что произойдёт с фразой «какие уроки»?
3. Проверьте, что `answer_from_cache` не асинхронная, а обычная функция. Почему она может быть «обычной», хотя обработчики `async`?

9.5. Итоги главы

- Роутер группирует обработчики; декоратор `@router.message(фильтр)` их регистрирует.
- `F.command.contains(...)` плюс `|` составляют читаемый фильтр одной константой.
- Порядок роутеров важен: `fallback` «всепрощающий» стоит последним.
- `answer_from_cache` отдаёт кеш мгновенно или запускает фоновое обновление.
- DI: обработчики декларируют зависимости, `Dispatcher` их подставляет.
- `response_timeout` — страховка: при любом сбое пользователь получит фразу.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Что делает `F.command.contains("задани")` и как `|` объединяет условия?
2. Почему `fallback_router` включён в цепочку последним?
3. В чём смысл `worker.refresh_now()` при преждевременном ответе?
4. Как DI помогает тестировать обработчики без сети?

Решения.

1. Фильтр «команда содержит подстроку "задани"» — его пройдут «задания», «заданий» и т. п. `|` — «или»: достаточно любого из условий, чтобы фильтр сработал.
2. Фильтры роутеров проверяются по очереди; `fallback` без фильтра обязан стоять в конце, иначе он перехватил бы все запросы раньше всех.
3. Чтобы следующий запрос уже получил данные: отвечаем «рановато», а воркер в фоне наполняет кеш — кнопка «далее» вернёт готовый текст.
4. Обработчик не создаёт зависимости: тест передаёт фейковый кеш и фейковый воркер, и вся логика проверяется без сети и без запуска сервера.

Глава 10. Дирижёр оркестра: настоящий веб-сервер `skill.py`

В предыдущих главах мы собрали по отдельности все инструменты: конфиг, чистую логику, сетевого клиента, кеш, воркер, роутеры. Но оркестр — это не набор инструментов, это когда кто-то говорит им, когда и как играть. Этот кто-то — `skill.py`: **единственный модуль, который знает, как устроены все остальные**, и склеивает их в работающее приложение.

В ЭТОЙ ГЛАВЕ:

- почему `skill.py` — это «фабрика», из которой рождается приложение;
- в каком порядке подключаются роутеры и почему порядок имеет значение;
- как хуки `@dp.startup()` / `@dp.shutdown()` управляют жизнью воркера;
- зачем нужны `@dp.timeout()` и `@dp.errors()` — «страховка от тишины»;
- как приложение приземляется на `aiohttp` и путь `/alice`;
- что делает строка `if __name__ == "__main__": main()`.

10.1. Приложение как функция

PYTHON

```
def create_app(config: Config) -> web.Application:
    cache = HomeworkCache()
    worker = PrefetchWorker(
        fetch=lambda: fetch_homework(
            config.sgo.login, config.sgo.password, config.sgo.school,
        ),
        cache=cache,
        interval=config.prefetch_interval,
    )

    dp = Dispatcher(
        response_timeout=4.0,
        cache=cache,
        worker=worker,
    )

    # Router order matters: start → homework → more → fallback
    dp.include_router(start_router)
    dp.include_router(homework_router)
    dp.include_router(more_router)
    dp.include_router(fallback_router)

    @dp.startup()
    async def on_startup() -> None:
        logger.info("starting prefetch worker")
        await worker.start()

    @dp.shutdown()
    async def on_shutdown() -> None:
        logger.info("stopping prefetch worker")
        await worker.stop()

    @dp.timeout()
    async def on_timeout(event) -> str:
        logger.warning("timeout processing update")
        return TIMEOUT_TEXT

    @dp.errors()
    async def on_error(event) -> str:
        logger.exception("unhandled error in handler")
        return ERROR_TEXT

    skill = Skill(skill_id=config.skill_id)
    request_handler = OneSkillAiohttpRequestHandler(dispatcher=dp, skill=skill)

    app = web.Application()
    request_handler.register(app, path="/alice")
    setup_application(app, dp, skill=skill)
```

```
app["dispatcher"] = dp # expose for tests
return app
```

Прочитаем сверху вниз — это план всего приложения:

1. **Строим кеш.** Помните: `HomeworkCache()` — единственный экземпляр для всего приложения. Он будет жить в Dispatcher, и через него обработчики получают результат.
2. **Строим воркер.** Передаём в него `fetch`-лямбду, обернутую над конфигом СГО. «Заготовки», знакомые из глав 2, 6, рождаются здесь, на самом верху, где известен конфиг.
3. **Строим Dispatcher.** С `response_timeout` и зависимостями `cache / worker`. Это «мозг» обработки сообщений.
4. **Подключаем роутеры по порядку.** Ещё раз: `start → homework → more → fallback`. Это порядок разрешения фильтров из главы 9.
5. **Хуки жизненного цикла** — декораторы `@dp.startup()` / `@dp.shutdown()`: при запуске приложения стартует воркер, при остановке — аккуратно останавливается.
6. **Обработчики крайних случаев** — `@dp.timeout()` (обработка заняла больше 4 с → фраза `TIMEOUT_TEXT`) и `@dp.errors()` (непойманное исключение в обработчике → фраза `ERROR_TEXT`). Оба возвращают текст, который Алиса произнесёт вместо молчания. Это страховка от «пустого эфира» для пользователя.
7. **Описание навыка для Яндекс.Диалогов** — `Skill(skill_id=config.skill_id)`. `skill_id` сверяется с платформой — какой навык к нам обращается.
8. **Создаём aiohttp-приложение** `web.Application()`. Вебхук вешается так: `request_handler.register(app, path="/alice")` ставит route на пути `/alice`, а `setup_application` связывает приложение с Dispatcher.
9. **Сохраняем dispatcher в самом приложении** — `app["dispatcher"] = dp`: доступ для тестов и внешних инструментов.

ТЕРМИН. Фабрика приложения — функция, которая получает внешние данные (конфиг) и возвращает готовый работоспособный объект. `create_app` — такая фабрика: один вход (`Config`), один выход (`web.Application`).

10.2. Жизненный цикл приложения

PYTHON

```
@dp.startup()
async def on_startup() -> None:
    logger.info("starting prefetch worker")
    await worker.start()

@dp.shutdown()
async def on_shutdown() -> None:
    logger.info("stopping prefetch worker")
    await worker.stop()
```

- **startup:** приложение поднялось → воркер делает первый `_refresh_once` (кеш моментально наполняется) и запускает периодический цикл. Пока работает сервер — кеш обновляется.

- **shutdown:** приложение гасится → `await worker.stop()` отменяет фоновую задачу и дожидается её завершения. Чистый выход, никаких «фантомных» работ после смерти.

Обратите внимание на `await` перед `worker.stop()`: остановка — тоже асинхронная операция (нужно убедиться, что цикл действительно завершился).

ВАЖНО. Хуки жизни живут **ВНУТРИ DISPATCHER**, а не в aiohttp-приложении. Воркер обязан стартовать **ДО** приёма первого запроса (иначе кто-то может спросить «что задали» при пустом кеше) и останавливаться **ПОСЛЕ** прекращения приёма запросов — `aliceio` гарантирует этот порядок.

10.3. Точка входа

PYTHON

```
def main() -> None:
    config = load_config()
    app = create_app(config)
    web.run_app(app, host=config.host, port=config.port)

if __name__ == "__main__":
    main()
```

- `load_config()` → `create_app()` → `web.run_app(...)`. Красиво и симметрично.
- `if __name__ == "__main__":` — эта строка срабатывает только когда файл запущен как **скрипт** (`python -m alice_skill.skill` или `python alice_skill/skill.py`). Если этот модуль просто импортируется другой частью программы, `main()` не вызывается. Это стандартная «дверь для запуска», отличающая `import` от запуска.

СОВЕТ. Правило «импорт не должен запускать приложение» критично для тестов: `test_skill.py` импортирует `create_app` и строит приложение из конфига-заглушки — и ничего не запускается само собой. Точка входа под контролем `main()`.

Хост и порт приходят из конфига (по умолчанию `127.0.0.1:8000`) — на проде за локальным адресом обычно стоит reverse проху (глава 12).

10.4. Что здесь не так «по-детски»

- Параметры в `create_app` — весь конфиг → все компоненты. Конфиг — интерфейс между флагманским входом и остальным миром.
- Внутри нет импорта секретов — `load_config` сама всё вытянула из JSON/env задолго до создания приложения.
- Приложение — почти чистая функция от конфига: его можно создавать в тестах сколько угодно и с любым конфигом (как делают тесты `test_skill.py`).

ПОПРОБУЙТЕ САМИ [🔧]

1. Запустите `python -m alice_skill.skill` с настоящим конфигом и откройте `http://127.0.0.1:8000/alice` в браузере. Что вернёт GET-запрос? (Подсказка: это webhook — он ждёт POST от Алисы.)
2. В `tests/test_skill.py` найдите, как тесты получают приложение: через `create_app(...)`. Что они подставляют вместо настоящего конфига?
3. Переставьте в коде местами `include_router(homework_router)` и `include_router(fallback_router)` и предскажите: что изменится в поведении навыка?

10.5. Итоги главы

- `skill.py` — единственная точка, которая знает всех и собирает приложение.
- Порядок роутеров фиксирован: `start` → `homework` → `more` → `fallback`.
- `@dp.startup()` / `@dp.shutdown()` управляют воркером строго до и после запросов.
- `@dp.timeout()` и `@dp.errors()` превращают любые сбои в текст для Алисы.
- `create_app` — чистая фабрика: из конфига рождается `web.Application`.
- `if __name__ == "__main__"` отделяет «запустили как скрипт» от «импортировали как модуль».

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Почему кеш и воркер создаются один раз, а не в каждом обработчике?
2. Что делают `@dp.startup()` / `@dp.shutdown()` и почему воркер стартует до webhook?
3. Что означает `if __name__ == "__main__": main()`?
4. Почему модуль называется `skill.py` и как связано `Skill(skill_id=...)` с навыком Алисы?

Решения.

1. Это суть принципа DI из главы 9 и залог консистентности: все обработчики читают один и тот же кеш и работают с одним воркером. Создание в каждом обработчике дало бы «разъехавшиеся» копии.
2. `startup` наполняет кеш и заводит периодический цикл до первого запроса; `shutdown` аккуратно гасит задачу. Это гарантирует, что webhook никогда не получит пустого кеша из-за недозапущенного воркера.
3. Код ниже этой строки выполняется только при запуске файла как скрипта, а не при импорте — точка входа отделена от модуля.
4. `Skill(skill_id=...)` объявляет платформе, какой навык обслуживает приложение; а имя модуля `skill.py` отражает его роль — сборку именно навыка.

Глава 11. Страховочная сетка: как устроены 78 тестов

Акробат выступает без страховочной сетки только в старых фильмах. В реальном цирке сетка есть всегда: она превращает ошибку в «ещё одну попытку», а не в катастрофу. Для нашего проекта с сетью и временем такая сетка — **78 тестов** в `tests/`. Это не роскошь, а страховка: тесты позволяют менять код, не боясь сломать логику. Разберём, чем тестируется проект.

В ЭТОЙ ГЛАВЕ:

- как запустить все тесты и читать отчёт;
- что такое фикстуры и почему здесь нет своего `conftest.py`;
- как моки с `спес=` проверяют поведение, ничего не запуская;
- как тестируют асинхронный код без сети;
- почему сетевой слой проверяют «фейком», а не реальной СГО;
- что такое цикл RED → GREEN → REFACTOR.

11.1. pytest: как это запускается

В `requirements.txt` есть `pytest>=8.0.0` и `pytest-asyncio>=0.21.0`. Первый — сам фреймворк тестов, второй — поддержка асинхронных тестов (`async def test_...`).

BASH

```
python -m pytest -v
```

pytest найдёт все файлы `test_*.py` и функции `test_*`. Каждая успешная — зелёная точка, каждая упавшая — отчёт со стеком **именно той строки, которая сломалась**. Имена тестов — мини-документация: `test_format_for_voice_all_week`, `test_sgo_empty_day` и так далее.

СОВЕТ. Флаг `-v` («verbose») печатает полные имена тестов. Гоняйте его хотя бы раз: тогда отчёт читается как список того, что умеет проект.

11.2. Подготовка сцены: фикстуры, хелперы и моки

Проект активно пользуется двумя **встроенными фикстурами** pytest — `tmp_path` (временный каталог; в него кладётся `config.json` — вы видели в главе 4) и `monkeypatch` (временная установка env-переменных). Общих кастомных фикстур в `conftest.py` нет — подготовку «сцены» делают маленькие функции-хелперы прямо в файле теста.

ТЕРМИН. *Фикстура* — это «подготовленная сцена» для теста: временная папка, временные переменные окружения, заранее заполненный объект. pytest создаёт и убирает фикстуру сам; тест лишь просит её аргументом.

В `tests/test_handlers.py` заготовки выглядят так:

```
def _ok_result(text: str = "На завтра одно задание.") -> HomeworkResult:
    return HomeworkResult(
        status="ok", target_date=datetime.date(2026, 9, 21), text=text,
    )

def _fake_cache(result) -> HomeworkCache:
    cache = HomeworkCache()
    if result is not None:
        cache.set(result)
    return cache

def _fake_worker() -> MagicMock:
    return MagicMock(spec=PrefetchWorker)

def _fake_message(command: str = "") -> MagicMock:
    msg = MagicMock()
    msg.command = command
    return msg
```

- `_ok_result()` — «фабрика правды»: валидный `HomeworkResult`.
- `_fake_cache` — кеш, в который уже положен нужный результат (или пустой).
- `_fake_worker() -> MagicMock(spec=PrefetchWorker)` — **мок** с интерфейсом воркера, но без реальной логики: мы можем проверить, что метод `refresh_now()` вызывался, ничего не запуская. `spec=` ограничивает мок полями/методами настоящего класса — ошибка в имени метода тут же подсветится.
- `_fake_message` — «сообщение»-заглушка, которому достаточно поля `command`.

Тест выглядит как «документ»:

```
@pytest.mark.asyncio
async def test_start_handler_returns_homework_from_cache():
    cache = _fake_cache(_ok_result("На завтра математика."))
    worker = _fake_worker()
    result = await handle_start(_fake_message(), cache=cache, worker=worker)
    assert result.text == "На завтра математика."
    worker.refresh_now.assert_not_called()
```

- Кеш заполнен → ответ пришёл из кеша.
- `worker.refresh_now.assert_not_called()` — важное **отрицательное** утверждение: раз ответ достался из кеша, фоновую подгрузку запускать не нужно.

ВОПРОСЫ ЛЮБОПЫТНЫХ. Зачем `spec=PrefetchWorker`, если у мока и так есть все методы?

Без `spec` мок «примет» любой атрибут, даже опечатку: `worker.refresh_no()` молча создаст фейковый вызов и тест может пройти мимо ошибки. С `spec` мок знает только настоящие поля класса — опечатка упадёт сразу. Это защита от «тихих» ложных зелёных тестов.

11.3. Асинхронные тесты

`pytest-asyncio` позволяет так:

PYTHON

```
from unittest.mock import AsyncMock

@pytest.mark.asyncio
async def test_refresh_now_spawns_background_task():
    cache = HomeworkCache()
    fetch = AsyncMock(return_value=_ok_result())
    worker = PrefetchWorker(fetch=fetch, cache=cache, interval=86400)

    await worker.start()
    await asyncio.sleep(0)

    fetch.reset_mock()
    worker.refresh_now()
    await asyncio.sleep(0)
    assert fetch.await_count == 1

    await worker.stop()
```

- `AsyncMock(return_value=...)` — готовая асинхронная «подделка» из `unittest.mock`: `await fetch()` мгновенно возвращает сфабрикованный результат. Сеть ни разу не касается теста.
- `await asyncio.sleep(0)` просит цикл событий сделать «шаг», чтобы фоновая задача успела отработать.
- `fetch.reset_mock()` — сброс счётчиков вызовов: дальше мы считаем только «хвост», то есть один дополнительный вызов из-за `refresh_now()`.
- `await worker.stop()` аккуратно останавливает периодический цикл.

11.4. Тесты для сетевого слоя: фейк вместо реальности

Настоящий тест `sgo.py` с живым сервером был бы невозможен: чужая система, нестабильный интернет, пароли. Поэтому сетевой слой тестируется **через заглушку-фейк**: подменяют клиент `NetSchoolAPI` на объект, который ведёт себя предсказуемо:

```

@pytest.mark.asyncio
async def test_fetch_homework_ok():
    mock_ns = AsyncMock()
    mock_ns.diary = AsyncMock(return_value=fake_diary)
    mock_ns.attachments = AsyncMock(return_value=[])
    mock_ns.logout = AsyncMock()

    with patch("alice_skill.sgo.NetSchoolAPI", return_value=mock_ns):
        result = await fetch_homework("u", "p", "s", now=datetime.date(2026, 9, 20))

    assert result.status == "ok"
    assert "алгебра" in result.text
    mock_ns.login.assert_awaited_once_with("u", "p", "s")
    mock_ns.logout.assert_awaited_once()

```

`patch("alice_skill.sgo.NetSchoolAPI", return_value=mock_ns)` — контекстный менеджер `unittest.mock.patch`: временно подменяет класс клиента на фейк, а после блока `with` подмена автоматически откатывается. Упомянутый в главе 4 `monkeypatch` делает то же, но в стиле `pytest-фикстур`. Сетевой слой ведёт себя с фейком как с реальным, потому что интерфейс один.

АНАЛОГИЯ. Это как автоинструктор на симуляторе: машина не настоящая, но руль, педали и правила те же. Ученик учится вождению, не рискуя машиной. Мы учим нашу логику взаимодействовать с СГО, не рискуя чужим школьным сервером.

Это важный навык: тестировать **свою логику**, а не чужие библиотеки. Нашу логику — формирование текста, выбор ответа по статусу, кеширование — тестируют. Сеть и работа чужих библиотек — нет: их мы не контролируем.

11.5. Танец RED → GREEN → REFACTOR

Представьте, что мы хотим, чтобы навык при «пустой» домашке говорил иначе. TDD (test-driven development) — процесс «тест сначала»:

1. **RED**: пишем тест на новую фразу → запускаем → он падает (функции ещё нет).
2. **GREEN**: пишем минимальный код функции → тест зелёный.
3. **REFACTOR**: улучшаем читаемость; тест зелёный каждый раз.

Красота в том, что тест фиксирует *поведение*, а не *реализацию*. Можно сколько угодно переписывать внутренности — поведение не изменится. Именно поэтому проект с 78 тестами меняется без страха.

11.6. Список тестов как документация

Быстрый взгляд на названия тестов рассказывает о поведении:

- `test_config...` — загрузка, env-приоритет, обязательные поля, значения по умолчанию.
- `test_homework...` — 15 тестов: склонения, падежи, пустая домашка, «игрушечные» списки.
- `test_cache...` — set/get, перезапись, свежесть (`is_stale`).
- `test_worker...` — старт/стоп, фоновая задача, ошибка `fetch`, защита от повторного входа.
- `test_sgo...` — ок/пусто/ошибка на фейковом `NetSchoolAPI`, вложения, гарантированный `logout`.

- `test_handlers...` — все роутеры и логика `answer_from_cache`.
- `test_skill...` — сборка приложения, порядок роутеров, DI, обработчики таймаута/ошибок.
- `test_encoding...` — весь русский источник проекта проверяется как валидный UTF-8 (защита от «квадратиков» и битых байтов в консоли).

ПОПРОБУЙТЕ САМИ [🔪]

1. Запустите `python -m pytest tests/test_encoding.py -v` — зелёные тесты подтверждают, что все источники в UTF-8.
2. Откройте `tests/test_handlers.py` и найдите «отрицательное» утверждение с `assert_not_called`. Какое поведение оно фиксирует?
3. Напишите тест по сценарию RED для новой фразы — например, что навык при пустом кеше отвечает `PREMATURE_TEXT`. Запустите и почувствуйте шаг RED.

11.7. Итоги главы

- 78 тестов — страховочная сетка, позволяющая менять код без страха.
- Встроенные фикстуры `tmp_path` и `monkeypatch` готовят «сцену» без своего `conftest.py`.
- Моки с `spec=` и отрицательные `assert_not_called` ловят «тихие» ошибки.
- `AsyncMock` и `await asyncio.sleep(0)` накатывают асинхронные тесты без сети.
- Сетевой слой тестируется фейком через `patch(...)`: интерфейс один, поведения правда.
- Цикл RED → GREEN → REFACTOR фиксирует поведение, а не реализацию.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Что такое фикстура и зачем pytest передаёт её в тест параметром?
2. Почему тесты не обращаются к настоящему СГО?
3. Опишите RED/GREEN/REFACTOR на примере любой новой фразы навыка.

Решения.

1. Фикстура — подготовленная «сцена»: временная папка, переменные окружения или готовый объект. pytest сам создаёт её до теста и убирает после; тест лишь объявляет фикстуру аргументом.
2. Настоящий СГО недоступен тестам: это чужая система с паролями и зависимостью от сети. Вместо неё подставляется фейк с тем же интерфейсом.
3. RED: пишем тест «нужная фраза в ответе» и видим, как он падает. GREEN: добавляем фразу в код, тест зеленеет. REFACTOR: улучшаем стиль — тест остаётся зелёным, потому что фиксирует поведение, а не текст реализации.

Глава 12. Городской адрес: как навык становится видимым для Алисы

127.0.0.1 — это как «дом без таблички»: в комнате всё работает, но никто на улице не знает, что вы здесь. Алиса — облачный «город», она не может зайти к вам в гости по локальному адресу. Чтобы навык стал видимым, нужен **публичный адрес**: витрина, вывеска, а за ней — вход в ту самую комнату на **127.0.0.1:8000**. Разберём путь от туннеля для разработки до боевого сервера с HTTPS.

В ЭТОЙ ГЛАВЕ:

- полная картина продакшена: от пользователя до aiohttp;
- туннель ngrok для быстрой проверки навыка у реальной Алисы;
- VPS, секреты в окружении и systemd вместо ручного запуска;
- reverse proxy nginx и обязательный HTTPS;
- чем смотреть на логи и проверять, что сервис жив.

12.1. Полная картина продакшена

Локально всё отлично работает на **127.0.0.1:8000** — но «тихо». Продакшен — это цепочка публичных звеньев:

TEXT

```
Пользователь → Яндекс.Диалоги (https://dialogs.yandex.ru)
                → ваш навык с webhook-URL
                → https://ваш-домен/alice
                  ↓
            reverse proxy (nginx)
                  ↓
            127.0.0.1:8000 (aiohttp, наш сервис)
```

Пользователь разговаривает с Алисой, Алиса отправляет запросы по webhook-URL, URL ведёт на ваш домен, домен — на nginx, а nginx бережно передаёт запрос внутреннему aiohttp. Каждое звено — отдельная маленькая задача, и каждое можно заменить, не трогая остальные.

12.2. Туннель для разработки

Самый быстрый способ «просочить» локальный сервер наружу — **ngrok** (или аналог: localtunnel, cloudflared):

BASH

```
ngrok http 8000
```

ngrok выдаёт публичный адрес вида **https://abc123.ngrok-free.app**. В личном кабинете Яндекс.Диалогов в поле **Webhook URL** вставляем **https://abc123.ngrok-free.app/alice** и нажимаем «Сохранить». Готово — навык принимает запросы.

Почему **/alice** на конце? Это путь, который мы назначили в `request_handler.register(app, path="/alice")` (глава 10): вебхук живёт по адресу **host:port/alice**.

ВАЖНО. Это «строительные леса» для проверки, а не фундамент. Адрес ngrok меняется при каждом запуске, туннель не вечен, а ваш локальный сервис оказывается доступен из интернета — под временным адресом и без HTTPS. Для разработки и презентации — находка; для боевого навыка — нет.

ТЕРМИН. *Reverse proxy* — сервис (обычно nginx), который принимает запросы «с улицы», а дальше проксирует их внутреннему приложению. Он может добавить HTTPS, балансировку и скрыть внутреннюю архитектуру.

12.3. Продакшен-развёртывание

Для настоящего боевого навыка ngrok не годится (адрес меняется, туннель не держится вечно). Нужен выделенный сервер. Минимальный путь:

1. **VPS** (любой: Hetzner, DigitalOcean, Timeweb...): установить Python, клонировать репозиторий, создать venv, `pip install -r requirements.txt`.
2. **Конфиг и секреты:** значения `SGO_LOGIN`, `SGO_PASSWORD` и другие положить в системные переменные окружения, а не в `config.json` — так секретов нет ни в git, ни в лишних файлах.

СОВЕТ. Секреты живут в окружении, потому что их нет в репозитории. Если пароль случайно попадёт в git-историю — он «светится» вечно: историю не переписать, а менять пароль придётся. Env-переменные легко задаются через systemd и не попадают в код.

3. **Запуск.** Самый простой вариант для проверки — запуск в фоне: `nohup python -m alice_skill.skill > /tmp/alice-homework.log 2>&1 &`. Для постоянной работы — systemd-юнит:

INI

```
[Unit]
Description=alice-homework skill
After=network.target

[Service]
User=alice
WorkingDirectory=/home/alice/alice-homework
EnvironmentFile=/home/alice/alice-homework/.env
ExecStart=/home/alice/alice-homework/.venv/bin/python -m alice_skill.skill
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

- `Restart=always` — сервис сам поднимется после сбоев.
- `EnvironmentFile` — секреты вне git, в файле `/home/alice/alice-homework/.env`.

АНАЛОГИЯ. `nohup ... &` — «пустить и забыть»: пока процесс жив, он работает; упал — лежит, пока кто-нибудь не заметит. `systemd` — «трезвый вахтёр»: заметил, что процесс упал, через 5 секунд поднял снова, а при перезагрузке сервера — поднял всё, что найдено в `WantedBy=multi-user.target`.

4. **Reverse proxy — nginx:** запросы «с улицы» принимает nginx, а он проксирует на `127.0.0.1:8000`. Плюс — TLS (HTTPS): Яндекс.Диалоги требуют HTTPS-адрес.

NGINX

```
server {
    listen 443 ssl;
    server_name alice.example.com;

    ssl_certificate      /etc/letsencrypt/live/alice.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/alice.example.com/privkey.pem;

    location /alice {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

ТЕРМИН. TLS (и его «бытовое» имя HTTPS) — шифрование трафика между клиентом и сервером. Яндекс.Диалоги не принимают webhook по открытому HTTP: данные пользователей не должны ходить в открытую. Сертификаты от Let's Encrypt — бесплатные и перевыпускаются автоматически.

5. **Публикация вебхука:** HTTPS-адрес nginx указываем в настройках навыка.

12.4. Режим отладки и мониторинг

- **Логи:** `journalctl -u alice-homework -f` — follow-хвост логов сервиса.
- **Готовность:** `curl` к вебхуку с фейковым JSON (глава 2) — быстрый smoke-тест, что сервис жив.
- `aiohttp` сам логирует HTTP-трафик; наши `logger.info(...)` из `sgo.py` расскажут, когда был поход в СГО и что вернулось.

ПОПРОБУЙТЕ САМИ [🔗]

1. Поднимите `ngrok http 8000` рядом с запущенным навыком и вставьте адрес с `/alice` в Яндекс.Диалоги. Спросите «что задали?» — и получите ответ через настоящий туннель.
2. Загляните в `journalctl -u alice-homework -f` и найдите запись о походе в СГО: `logger.info(...)` из `sgo.py` работает как «шпион» на службе.
3. Представьте, что nginx отдаёт на `127.0.0.1:8000` весь трафик, а не только `/alice`. Какая поверхность атаки появится?

12.5. Итоги главы

- `127.0.0.1:8000` — «дом без таблички»; для Алисы нужен публичный адрес.
- `ngrok` — «строительные леса»: быстро проверить навык, но не для боевого использования.

- Секреты — в переменные окружения, никогда в git.
- systemd с `Restart=always` присматривает за процессом в продакшене.
- nginx принимает HTTPS-запросы и проксирует их на aiohttp; HTTPS обязателен.
- Логи (`journalctl` , aiohttp, `logger.info`) отвечают на вопрос «что произошло».

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Зачем нужен ngrok на ранней стадии и почему он не для продакшена?
2. Зачем HTTPS обязателен для webhook Алисы?
3. Что делает `Restart=always` в systemd?

Решения.

1. ngrok мгновенно даёт публичный адрес локального сервера — идеально для проверки у реальной Алисы. Для продакшена он не подходит: адрес меняется, туннель не вечен, а навык должен жить стабильно.
2. Яндекс.Диалоги требуют HTTPS: данные пользователей идут по зашифрованному каналу. Сертификат получают через Let's Encrypt, а обслуживает его nginx.
3. `Restart=always` велит systemd автоматически поднимать сервис после любого сбоя. С `RestartSec=5` подъём произойдёт через 5 секунд.

Глава 13. Не финиш, а перекрёсток: куда можно вырасти

Проект небольшой, но живой. Вопрос «что дальше» — отличный учебный полигон: в нём уже есть сеть, кеш, фоновые задачи, DI и тесты — всё, что нужно, чтобы попробовать что-то серьёзное. Перечислим направления по возрастанию сложности: где просто «докрутить», а где — начать следующий проект.

В ЭТОЙ ГЛАВЕ:

- многопользовательский режим и Telegram-бот на том же webhook;
- богаче контент: дата по запросу, оценки, любимые предметы;
- диалог с памятью: FSM и `FiniteStateMachine`;
- харденинг: таймауты, `retry`, Prometheus;
- A/B-фразы и генеративный ответ на базе LLM.

13.1. Многошкольность и чат-боты

- Сейчас один кеш и один пользователь. Можно сделать многопользовательский режим с привязкой «`user_id` → школа/логин». Кеш превратится в `dict[user_id, HomeworkResult]` или в Redis.
- Тот же webhook легко переиспользовать для Telegram-бота (тот же `aliceio`-механизм, только библиотека для TG).

АНАЛОГИЯ. Сейчас кеш — одна полка под один сэндвич. Многопользовательский режим — шкафчик с ячейками: у каждого пользователя своя ячейка, и ключ — его `user_id`.

13.2. Богаче контент

- По запросу «на какой день» — выбор конкретной даты, неделя вперёд.
- Оценки (`assignment.score`), что «сдано»/«не сдано».
- Персональные предметы и уровни («только алгебра и физика»).

13.3. Состояние диалога: FSM

Сейчас навык без памяти — всё работает «здесь и сейчас». Настоящий диалог («Алиса, что задали?» — «На какой день?» — «На среду») требует **FSM** (конечных автоматов): у каждого сеанса есть состояние («жду день недели» и т.п.) и переходы между ними. В `aliceio` для этого есть `FiniteStateMachine`. Хорошая задача для следующего проекта.

ТЕРМИН. *FSM* (finite state machine, конечный автомат) — модель, где у разговора есть конечный набор состояний и правила перехода между ними. Пока навык «ждёт день недели», команды вроде «покажи алгебру» трактуются как ответ на вопрос, а не как новая команда.

13.4. Харденинг и метрики

- Таймауты на каждый сетевой вызов (их мы уже закладывали — `response_timeout`).
- `Retry` с экспоненциальной задержкой для «СГО упало».
- Метрики (`/metrics` Prometheus): время ответа, число запросов, состояние кеша.

13.5. А/В-фразы и генеративный ответ

- Варианты открывалок и прощаний подтягивать из конфига — тот же `format_for_voice` с `rng`.
- Соединить с большой языковой моделью: навык сам перескажет домашку «человеческим» голосом (по сути — замена части `homework.py` на вызов LLM).

ПОПРОБУЙТЕ САМИ [🔗]

1. Нарисуйте диаграмму FSM для диалога «что задали? → на какой день? → на среду». Сколько состояний и переходов получилось?
2. Придумайте, какие метрики для `test_cache.py`-подобного отчёта были бы полезны: «свежесть», «число промахов», «среднее время ответа» — и как их считать.
3. Выберите один маршрут из этой главы и сформулируйте его как задачу с критерием готовности. Три строки — достаточно.

13.6. Итоги главы

- Многопользовательский режим превращает кеш в шкафчик с ячейками.
- Тот же webhook годится для Telegram-бота — `aliceio`-механизм общий.
- Диалог с памятью требует FSM; `FiniteStateMachine` уже есть в `aliceio`.
- Харденинг: таймауты, `retry`, метрики Prometheus — путь от «работает дома» к «живёт в проде».
- А/В-фразы и LLM-пересказ делают голос навыка живее.

ПРОВЕРЬТЕ СЕБЯ. Сначала ответьте по памяти, потом сравните с решениями.

1. Что изменится в кеше, если пользователей станет двое?
2. Зачем FSM навыку, который отвечает «здесь и сейчас»?
3. Почему `retry` с экспоненциальной задержкой лучше мгновенных повторных попыток?

Решения.

1. Кеш перестанет быть одним неразделяемым объектом: понадобится ключ `user_id`, чтобы у каждого был свой ответ. Простой вариант — `dict[user_id, HomeworkResult]`, дальше — Redis и TTL.
2. FSM нужен, когда между репликами пользователя есть смысл: «на какой день?» после «что задали?» — это ответ, а не новая команда. Без состояния такой диалог не построить.
3. Мгновенный повтор при упавшем СГО лишь нагружает упавший сервис. Растущая пауза даёт СГО время «прийти в себя» и снижает нагрузку в моменты сбоя.

Глава 14. Глоссарий и шпаргалка

Любое большое путешествие заканчивается двумя вещами: картой и рюкзаком. Эта глава — и карта, и рюкзак: полный словарь понятий, встреченных в учебнике, и набор команд, которые пригодятся, когда вы сядете за код заново.

В ЭТОЙ ГЛАВЕ:

- словарь: от HTTP-запроса до `fail-fast` ;
- шпаргалка команд: окружение, запуск, тесты, туннель;
- чек-лист «я усвоил проект».

14.1. Словарь понятий

Пользуйтесь таблицей, когда читаете код: каждое понятие здесь — не из учебника, а из нашего проекта.

Термин	Значение в нашем проекте
HTTP-запрос	Текст, который Алиса шлёт на <code>/alice</code> : метод, путь, заголовки, JSON-тело
webhook	URL, куда внешняя система сама шлёт POST при событии
aiohttp	Асинхронный HTTP-фреймворк; наш веб-сервер
aliceio	Библиотека для Яндекс.Диалогов поверх aiohttp; роутеры, фильтры, Dispatcher
Dispatcher	Центральный «мозг»: берёт запрос, раздаёт по роутерам, собирает ответ
Router	«Коробка» обработчиков с фильтрами
F	Объект фильтров aliceio: <code>F.command.contains(...)</code> , <code>F.session.new</code>
<code>await</code>	Точка ожидания в корутине; цикл событий тем временем делает другие работы
корутина	Функция <code>async def</code> ; запускается через <code>await</code> или <code>asyncio.create_task</code>
asyncio	Стандартная библиотека асинхронного кода
dataclass	Класс, который хранит данные; <code>@dataclass(frozen=True)</code> — неизменяемый
DI (внедрение зависимостей)	Объекты-зависимости (cache, worker) передаются в обработчики Dispatcher'ом
кеш (кэш)	Быстрый буфер результатов; у нас — в памяти, один результат для всех
TTL / is_stale	«Время жизни» кеша; проверка свежести через <code>_fetched_at</code>
фикстура	Подготовка теста; pytest передаёт её значение в параметры
<code>monkeypatch</code>	Инструмент pytest: временная замена (env-переменные, атрибуты)
фейк-заглушка	Объект-заменитель сети/базы для теста, ведёт себя предсказуемо
TDD	«Тест сначала»: RED → GREEN → REFACTOR
graceful degradation	Работа при частичном сбое (вложения не получились — отвечаем без них)
reverse proxy	nginx перед aiohttp: HTTPS, перенаправление на <code>127.0.0.1:8000</code>
systemd	Менеджер сервисов Linux; <code>Restart=always</code> поднимает сервис после сбоя
env-переменная	Окружение процесса; <code>SGO_LOGIN</code> и др.; безопасное место для секретов
fail-fast	«Упади громко и рано»: проверки в <code>load_config</code> на старте, а не позже

ТЕРМИН. *fail-fast* — принцип «упади громко и рано». Наш `load_config` проверяет обязательные поля до запуска сервера: опечатка в имени переменной окружения упадёт на старте, а не «тихо под корнем в продакшене».

14.2. Мини-шпаргалка команд

BASH

```
# окружение и установка
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # Linux/macOS
pip install -r requirements.txt

# запуск
cp config.example.json config.json
python -m alice_skill.skill

# проверка вебхука без Алисы
curl -X POST http://127.0.0.1:8000/alice \
  -H "Content-Type: application/json" \
  -d '{"session":{"new":true,"session_id":"1","user_id":"1"},"version":"1.0","request":{"command":"что задали","type":"SimpleUtterance"}}'
```

```
# тесты
python -m pytest -v

# туннель
ngrok http 8000
```

14.3. Чек-лист «Я усвоил проект»

Пройдитесь и отметьте, что можете объяснить вслух без подглядывания:

- ☐ Какой путь проходит запрос Алисы от облака до кеша и обратно.
- ☐ Почему кеш и воркер существуют отдельно от webhook.
- ☐ Как `lambda` связывает конфиг и воркер.
- ☐ Что делает `try/except/finally` в `fetch_homework`.
- ☐ Зачем `frozen=True` в конфиге и `Literal` в статусе.
- ☐ Почему `homework.py` можно протестировать без сети.
- ☐ Как Диспетчер подставляет `cache` и `worker` в обработчики.
- ☐ В каком порядке включаются роутеры и почему это важно.
- ☐ Как тесты покрывают сеть «фейками», а логику — «игрушечными» данными.
- ☐ Как развернуть навык локально и на VPS.

14.4. Итоги главы

- Словарь — карта проекта: каждое понятие из таблицы живёт в нашем коде.
- Шпаргалка команд — рюкзак: окружение, запуск, curl-проверка, тесты, туннель.

- Чек-лист — зеркало: если все пункты объясняются вслух, проект усвоен.
-

Заключение

Мы прошли путь от webhook до кеша и воркера, от «грязного» сетевого слоя до «чистой» логики текста, от локального запуска до nginx и systemd. За обычной фразой Алисы — «Скажи домашку» — стоит решение нескольких задач, которые вы теперь умеете читать и воспроизводить: разделение ответственностей, асинхронность, кеширование, конфигурация без секретов, тестирование и деплой.

Главное, что стоит вынести: **проект — это не «перевод задания на Python».** Проект — это выбор границ между частями и дисциплин, которые держат эти границы. Код любой сложности читается, когда знаешь, где искать.

В следующий проект добавляйте по одному приёму из этого учебника — от печатного конфига до тестов-фикстур — и каждое новое маленькое правило будет добавлять вам уверенности, которую не даёт «просто работающий скрипт».

Удачи с уроками — и удачи с кодами!